

Online Appendices

Table of contents

Preface	2
Appendix A - Simulation Code and Parameters	3
1 Imports	3
2 Simulation Code	3
3 Main Simulation Parameters	8
Appendix B - Robustness Check of the Simulation	11
Appendix C - Replication of Study 1 with Experienced Decision-Makers	14
1 Preamble: Data Loading and Wrangling	14
2 Sample	15
3 Results	15
Appendix D - Replication of Study 1 with a Diagnostic Cue	20
1 Preamble: Data Loadings and Wrangling	20
2 Sample	22
3 Pre-Registered Dependent Variables	22
4 Model: Cue by Condition Interactions	24
5 Summary	31
Appendix E - Analysis of Participants' Choices in Study 2	32
1 Preamble: Data Loadings and Wrangling	32
2 Model: Cue by Condition Interactions	32
Appendix F - Full Results for Study 3b	40
1 Preamble: Data Loadings and Wrangling	40
2 Analysis of Choices During the Game	41
3 Survey Dependent Variables	49
Appendix G - Full Results for Study 3c	52
1 Preamble: Data Loadings and Wrangling	52
2 Analysis of Choices During the Game	53
3 Survey Dependent Variables	61
Appendix H - Analysis of Testing Strategies in Study 4	65
1 Preamble: Data Loading and Wrangling	65
2 Analysis of Testing Strategies	66
3 Further Evidence of Change in Testing Strategies	69
4 Belief Regarding the Slope	75
5 Slope Beliefs by Prompt Type (Increasing Condition)	76

Preface

This document contains the online appendices for the paper. Each appendix is a self-contained analysis with reproducible code. Click “Show Code” to expand any code block.

Appendix	Description
A	Simulation Code and Parameters
B	Robustness Check of the Simulation
C	Replication of Study 1 with Experienced Decision-Makers
D	Analysis of Participants’ Choices in Study 1 (Diagnostic Cue Replication)
E	Analysis of Participants’ Choices in Study 2
F	Full Results for Study 3b
G	Full Results for Study 3c
H	Analysis of Testing Strategies in Study 4

Appendix A - Simulation Code and Parameters

This appendix presents the simulation code and parameters used to generate the results discussed in the paper. The full code is also available on the OSF repository of the project: https://osf.io/unp6a/?view_only=bc12e7b98b004a69a194dec39c2f3d41.

1 Imports

```
import warnings
from itertools import product

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import scipy.stats as stats
import seaborn as sns

from scipy.signal import detrend

warnings.filterwarnings("ignore")
```

2 Simulation Code

2.1 Agent

The **Agent** class represents a learner who updates beliefs about cue-outcome relationships across rounds. On each round, the agent:

1. Chooses a prospect according to its choice strategy (positive test or random).
2. Observes the payoff and compares it to the average of past payoffs.
3. Updates beliefs: if the payoff exceeds expectations, the agent strengthens beliefs in the direction of the chosen prospect's cue values (relative to the unchosen alternatives); if the payoff falls below expectations, the agent weakens those beliefs.

```

class Agent(object):
    """
    A learning agent.
    """

    def __init__(
        self, name, init_belief,
        random_choice_prob=0, payoffs_type="Increasing"
    ):
        self.name = name
        self.current_beliefs = init_belief
        self.past_beliefs = []
        self.random_choice_prob = random_choice_prob
        self.past_payoffs = []
        self.current_round = 0
        self.payoffs_type = payoffs_type

    def __repr__(self):
        return f"""Agent {self.name} at Round {self.current_round}.
        Current Beliefs: {self.current_beliefs}
        """

    def compute_utility(self, prospects):
        """
        Compute the utility of a set of prospects, based on current beliefs.
        """
        utility = np.dot(prospects, self.current_beliefs)
        return utility

    def compute_choice_probabilities(self, prospects):
        """
        Compute the choice probability of a set of prospects,
        based on current beliefs.
        """
        X = self.compute_utility(prospects)
        eXB = np.exp(X)

```

```

        return (eXB / eXB.sum(0)).flatten(order="F")

def choose_prospect(self, prospects, choice_strategy="Positive"):
    """
    Chooses a prospect among a set, based on a specified choice strategy
    ('Positive' or 'Random').
    """
    n_options = prospects.shape[0]
    options = np.arange(n_options)
    if choice_strategy == "Positive":
        p = self.compute_choice_probabilities(prospects)
    else:
        p = [1 / n_options] * n_options
    return np.random.choice(options, p=p)

def advance_to_next_round(self, prospects, payoff):
    """
    Advance the agent to the next round:
    * Selects a prospect among a set
    * Compares the revealed payoff of the prospect to expectations
    * Update the beliefs, and saves the current beliefs/payoff.
    """
    if np.random.uniform() > self.random_choice_prob:
        choice = self.choose_prospect(
            prospects, choice_strategy="Positive"
        )
    else:
        choice = self.choose_prospect(
            prospects, choice_strategy="Random"
        )
    # Compute the difference between attributes
    # of chosen and non-chosen prospects.
    chosen_prospect = prospects[choice, :]
    other_prospects = np.delete(prospects, choice, axis=0)
    diff_prospects = chosen_prospect - other_prospects.mean(axis=0)
    if self.current_round > 0:

```

```

        # Compare payoff to expectations
        positive_payoff = (
            1 if (payoff > np.mean(self.past_payoffs)) else -1
        )
        # Add difference to beliefs if payoff > expectations,
        # otherwise subtract it.
        self.current_beliefs += (
            diff_prospects * positive_payoff * 0.25
        )
        self.past_payoffs.append(payoff)
        self.past_beliefs.append(self.current_beliefs.copy())
    else:
        self.past_payoffs.append(payoff)
        self.past_beliefs.append(self.current_beliefs.copy())
    self.current_round += 1

```

2.2 Learning Environment

The `LearningEnvironment` class manages the simulation: it holds the prospect configurations for each round, the payoff sequences for each condition, and the population of agents. Calling `simulate()` runs all agents through all rounds.

```

class LearningEnvironment(object):
    """
    A learning environment is defined by:
    * A set of prospects
    * Several possible deterministic set of payoffs that the agent
      will receive over the multiple rounds
    * A set of agents
    """

    def __init__(self, prospects=None, payoffs=None):
        self.prospects = prospects
        self.n_rounds = prospects.shape[0]
        self.payoffs = payoffs
        self.has_simulated = False
        self.agents = []

```

```

def add_agents(self, agents):
    """
    Add a list of agents to the learning environment.
    """
    if self.has_simulated:
        raise Exception(
            "Agents cannot be added after the simulation "
            "has completed."
        )
    else:
        for agent in agents:
            self.agents.append(agent)

def simulate(self):
    """
    Simulate the behavior of the agents in the learning environment.
    """
    for agent in self.agents:
        for prospect, payoff in zip(
            self.prospects, self.payoffs[agent.payoffs_type]
        ):
            agent.advance_to_next_round(prospect, payoff)
    self.has_simulated = True

def get_end_beliefs(self):
    """
    Get the final beliefs of agents in a dataframe.
    """
    if not self.has_simulated:
        raise Exception(
            "The simulation needs to run before beliefs "
            "can be returned!"
        )
    df_beliefs = pd.DataFrame(
        [ag.name for ag in self.agents],

```



```

        columns=["AgentName"],
    )
    beliefs = np.array(
        [ag.current_beliefs for ag in self.agents], dtype=float
    )
    for i, b in enumerate(beliefs.T):
        df_beliefs[f"Belief_{i+1}"] = b
    return df_beliefs

```

3 Main Simulation Parameters

3.1 Alternatives and Cues

The simulation uses $J = 3$ alternatives defined by $p = 6$ cues, over $N = 20$ rounds. Cue values are drawn from a standard normal distribution and are independent of the payoffs.

```

J = 3    # Number of alternatives
N = 20   # Number of rounds
p = 6    # Number of cues

np.random.seed(28390312)

# Generating random prospects:
cue_values = np.random.normal(0, 1, size=(J * N, p))

```

3.2 Payoffs

The payoff sequences are deterministic and identical for all agents within a condition. They are designed so that the average payoff increases, stays flat, or decreases over the 20 rounds, while the round-to-round variability is matched across conditions.

```

payoffs_game = {
    "Increasing": [
        4.53, 2.40, 3.87, 3.40, 5.10, 4.57, 7.23, 5.03, 4.93, 7.13,
        7.63, 7.37, 7.17, 7.37, 10.10, 8.33, 8.47, 9.60, 9.30, 9.13,
    ],
    "Flat": [

```

```

        7.87, 5.37, 6.50, 5.70, 7.03, 6.17, 8.47, 5.90, 5.47, 7.30,
        7.47, 6.87, 6.30, 5.83, 8.57, 6.43, 6.23, 7.00, 6.37, 5.83,
    ],
    "Decreasing": [
        11.20, 8.37, 9.13, 7.97, 8.97, 7.73, 9.73, 6.80, 6.00, 7.50,
        7.30, 6.37, 5.43, 4.33, 7.00, 4.53, 3.97, 4.40, 3.40, 2.53,
    ],
}

```

3.3 Agents

For each of the 3 payoff conditions (Increasing, Flat, Decreasing) and 3 choice strategies (100% Positive Test, Mixed, 100% Random), the simulation creates **1,000 agents**, for a total of 9,000 agents. All agents start with null beliefs (a zero vector of length p).

```

N_AGENTS_EACH_TYPE = 1000
PAYOFFS_TYPES = ["Increasing", "Flat", "Decreasing"]
RANDOM_CHOICE_PROBS = [0, 0.5, 1]
INITIAL_BELIEFS = [0, 0, 0, 0, 0, 0]

agents_properties = product(
    PAYOFFS_TYPES,
    RANDOM_CHOICE_PROBS,
    np.arange(0, N_AGENTS_EACH_TYPE),
)

agents = [
    Agent(
        f"{payoff_type}_{random_choice_prob}_{i}",
        INITIAL_BELIEFS.copy(),
        random_choice_prob=random_choice_prob,
        payoffs_type=payoff_type,
    )
    for (payoff_type, random_choice_prob, i) in agents_properties
]

```

3.4 Running the Simulation

```
# Reshaping the alternatives for the simulation.
prospects_game = (
    df_alternatives.set_index(["round", "alternative"])[cue_columns]
    .values.reshape(-1, J, p)
    .astype(float)
)

# Setting up learning environment
learning_env_game = LearningEnvironment(
    payoffs=payoffs_game, prospects=prospects_game
)
learning_env_game.add_agents(agents)
learning_env_game.simulate()
```

3.5 Computing Belief Strength

After the simulation, we recover each agent's final beliefs and compute the average absolute belief strength across all cues. This measure captures the overall strength of the agent's (spurious) convictions about cue-outcome relationships.

```
df_end_beliefs = learning_env_game.get_end_beliefs()
df_end_beliefs["Trend"] = df_end_beliefs.AgentName.apply(
    lambda x: x.split("_")[0]
)
df_end_beliefs["Choice_Strategy"] = df_end_beliefs.AgentName.apply(
    lambda x: x.split("_")[1]
).map({"0": "100% Positive Test", "0.5": "Mixed", "1": "100% Random"})

df_end_beliefs["Strength_Beliefs"] = (
    df_end_beliefs[[f"Belief_{i}" for i in range(1, 7)]]
    .abs()
    .mean(axis=1)
)
```

Appendix B - Robustness Check of the Simulation

```
library(tidyverse)
library(here)

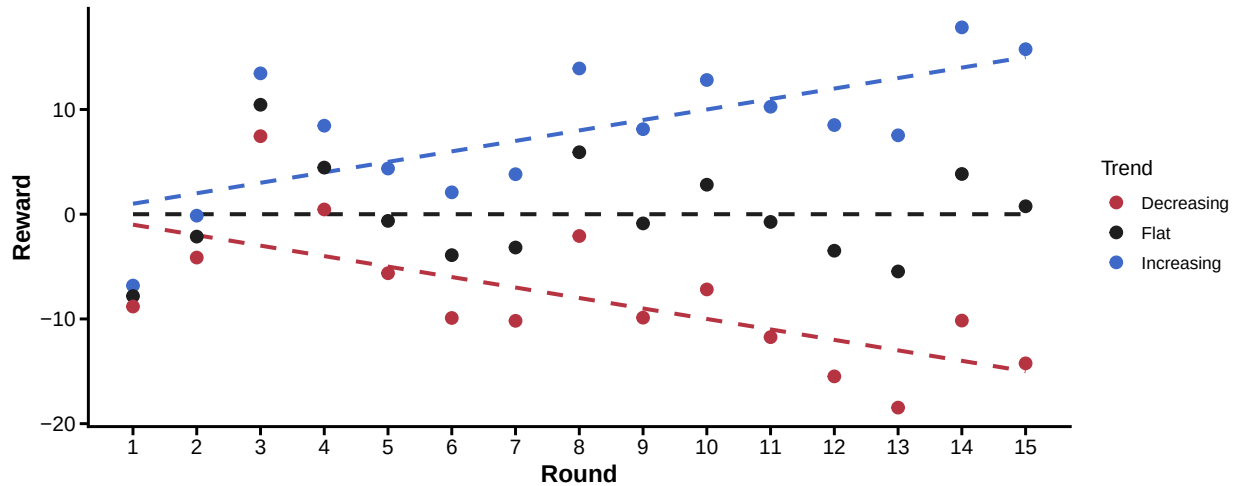
i_am("Appendices/AppendixB.qmd")
source(here("custom_theme.R"))
df <- read_csv(here("Simulation", "Simulation_Results_RobustnessCheck.csv"))
df_payoffs <- read_csv(
  here("Simulation", "Simulation_Payoffs_RobustnessCheck.csv")
)
```

This notebook shows the results of an alternative simulation with:

- $J = 5$ alternatives, defined by $p = 10$ cues
- Running for $N = 15$ rounds
- ... with random payoffs and attribute values.

0.1 Payoffs

```
ggplot(df_payoffs, aes(x = Round, y = Payoff, color = Trend)) +
  geom_smooth(
    method = "lm", se = FALSE, linetype="dashed",
    show.legend = FALSE
  ) +
  geom_point(size=3) +
  scale_x_continuous(breaks = 1:20) +
  labs(y = "Reward") +
  scale_color_manual(values = c("#b63441", "#1f1e1e", "#3f69c9")) +
  theme_matplotlib()
```



0.2 Results

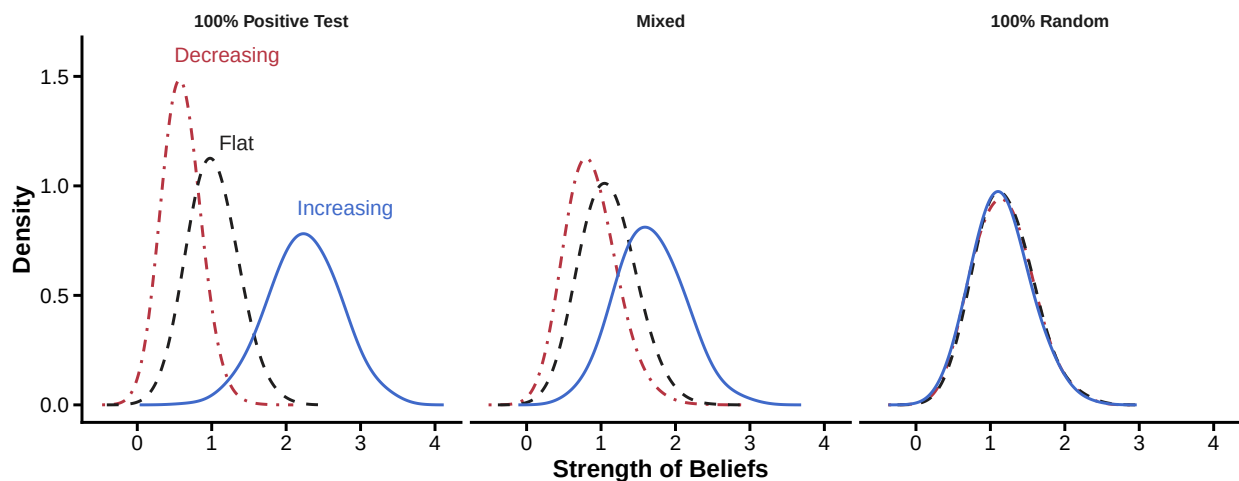
```
densities <- df %>%
  group_split(Choice_Strategy, Trend) %>%
  lapply(function(df) {
    dens <- density(df$Strength_Beliefs, bw=.2) # Compute density
    data.frame(x = dens$x, y = dens$y,
               Choice_Strategy = unique(df$Choice_Strategy),
               Trend = unique(df$Trend)) # Convert to data frame
  }) %>%
  bind_rows() %>%
  mutate(Choice_Strategy = factor(
    Choice_Strategy,
    levels=c("100% Positive Test", "Mixed", "100% Random"))
  )

annotation_df <- data.frame(
  x = c(.5, 1.1, 2.15),
  y = c(1.6, 1.2, .9),
  Trend = c("Decreasing", "Flat", "Increasing"),
  Choice_Strategy = factor(
    c("100% Positive Test", "100% Positive Test", "100% Positive Test"),
    levels = c("100% Positive Test", "Mixed", "100% Random")
  )
)
```

```

densities %>% ggplot(aes(x=x, y=y, color=Trend, linetype = Trend)) +
  geom_line(linewidth=.8) +
  scale_color_manual(values = c("#b63441", "#1f1e1e", "#3f69c9")) +
  facet_wrap(Choice_Strategy~.) +
  theme_matplotlib() +
  scale_y_continuous() +
  scale_linetype_manual(values=c("dotdash", "dashed", "solid")) +
  labs(x="Strength of Beliefs", y="Density") +
  geom_text(data = annotation_df,
            aes(x=x, y=y, label = Trend, color=Trend),
            inherit.aes = FALSE,
            hjust=0) +
  guides(color="none", linetype="none")

```



Appendix C - Replication of Study 1 with Experienced Decision-Makers

We conducted a conceptual replication of Study 1 as part of a classroom exercise with senior executives enrolled in “data-driven decision-making” programs at major European business schools in February, March, and September 2022.

Participants played the same investment simulation as in Study 1, with one modification: the design included only the Increasing and Decreasing conditions (the Flat condition was dropped). After playing 20 rounds, participants completed the same post-game survey measuring their subjective sense of learning, confidence in their attribute judgments, and the extremity of their predictions.

1 Preamble: Data Loading and Wrangling

```
library(tidyverse)
library(Hmisc)
library(effsize)
library(glue)
library(gt)
library(here)

i_am("Appendices/AppendixC.qmd")
source(here("custom_theme.R"))

survey_data <- read_csv(
  here("Studies/Appendix_Exec", "Data", "survey_clean.csv")
)

items <- c(
  "SubjInfluence",
  "EdgeOver",
  "BecameConfident",
  "CouldPredict",
  "AttribDiff",
```

```

    "NotEnoughRound",
    "N_Correct_Responses",
    "Confidence",
    "Extremity_Bets"
  )

  labels <- c(
    "Pred. Influence on ROI",
    "Pred. Edge over New Player",
    "Confident in Ability to Predict ROI",
    "Could Predict ROI by End Game",
    "Some Attrib. Were More Predictive",
    "Not Enough Rounds to Learn",
    "# Correct Attributes Comparisons",
    "Confidence in Attributes Comps.",
    "Extremity of Bets"
  )

```

2 Sample

```

n_total <- nrow(survey_data)
n_per <- survey_data %>% count(Condition)

```

The final sample consisted of 31 executives (15 in the Increasing condition, 16 in the Decreasing condition).

3 Results

```

# Compute t-tests and effect sizes for all DVs
results <- map_dfr(seq_along(items), function(i) {
  s <- items[i]
  formula <- as.formula(paste(s, "~ Condition"))
  tt <- t.test(formula, data = survey_data)
  cd <- cohen.d(formula, data = survey_data)
})

```



```

# t.test alphabetical: Decreasing - Increasing.
# Negate for Increasing - Decreasing.
tibble(
  DV = labels[i],
  t = -tt$statistic,
  df = tt$parameter,
  p = tt$p.value,
  d = -cd$estimate,
  d_ci_lower = -cd$conf.int[2],
  d_ci_upper = -cd$conf.int[1]
)
})

```

3.1 Survey Measures

```

# Compute t-stats for annotation
df_results <- results %>%
  mutate(
    formatted_p = ifelse(p < .001, "p < .001", glue("p = {round(p, 3)}")),
    label = glue(
      "t({round(df, 0)}) = {round(t, 3)}",
      "\n{formatted_p}\nd = {round(d, 2)}"
    ),
    item = factor(labels, levels = labels)
  )

# Pivot to long form
survey_data_long <- survey_data %>%
  pivot_longer(cols = all_of(items), names_to = "item") %>%
  dplyr::select(ResponseId, Condition, item, value) %>%
  mutate(item = factor(item, levels = items, labels = labels))

# Compute y-position for annotations
df_results <- df_results %>%
  left_join(
    survey_data_long %>%

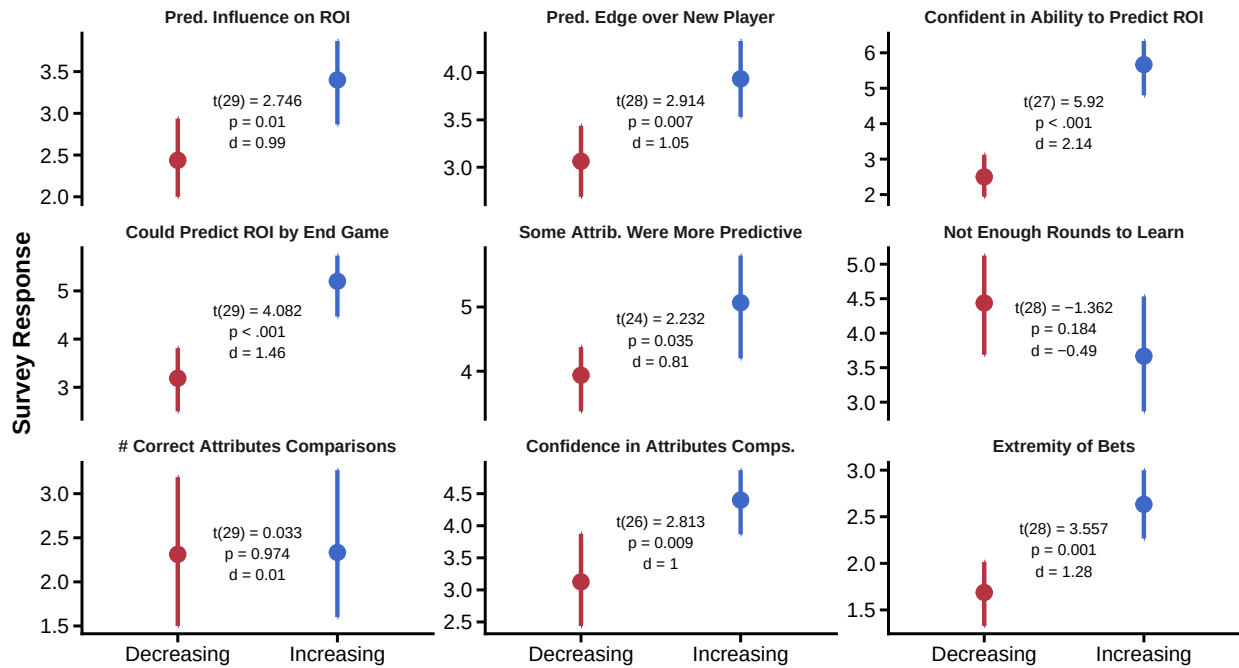
```

```

    group_by(item) %>%
    summarise(y_pos = mean(value, na.rm = TRUE), .groups = "drop"),
  by = "item"
)

ggplot(survey_data_long, aes(x = Condition, y = value, color = Condition)) +
  facet_wrap(~item, scales = "free_y") +
  stat_summary(
    fun = mean, geom = "point",
    position = position_dodge(width = 0.5), size = 4
  ) +
  stat_summary(
    fun.data = mean_cl_boot, geom = "errorbar", width = 0,
    position = position_dodge(width = 0.5), show.legend = FALSE,
    linewidth = 1.2
  ) +
  theme_matplotlib() +
  scale_color_manual(values = c("#b63441", "#3f69c9")) +
  labs(
    x = element_blank(),
    color = element_blank(),
    y = "Survey Response"
  ) +
  guides(color = "none") +
  geom_text(
    data = df_results,
    mapping = aes(x = 1.5, y = y_pos, label = label),
    inherit.aes = FALSE,
    color = "black",
    size = 3
  )

```



3.2 Statistical Results

```

results %>%
  mutate(
    d_ci = glue("[{round(d_ci_lower, 2)}, {round(d_ci_upper, 2)}]")
  ) %>%
  dplyr::select(DV, t, df, p, d, d_ci) %>%
  gt() %>%
  cols_label(
    DV = "Dependent Variable",
    t = md("*t*"),
    df = md("*df*"),
    p = md("*p*"),
    d = md("*d*"),
    d_ci = "95% CI"
  ) %>%
  fmt_number(columns = c(t, df), decimals = 1) %>%
  fmt_number(columns = d, decimals = 2) %>%
  fmt(
    columns = p,
    fns = function(x) ifelse(x < .001, "< .001", sprintf("%.3f", x))
  )

```

Independent-Samples t-Tests: Increasing vs. Decreasing Condition

Dependent Variable	<i>t</i>	<i>df</i>	<i>p</i>	<i>d</i>	95% CI
Pred. Influence on ROI	2.7	28.8	0.010	0.99	[0.21, 1.77]
Pred. Edge over New Player	2.9	27.9	0.007	1.05	[0.27, 1.84]
Confident in Ability to Predict ROI	5.9	26.9	< .001	2.14	[1.22, 3.06]
Could Predict ROI by End Game	4.1	29.0	< .001	1.46	[0.64, 2.29]
Some Attrib. Were More Predictive	2.2	23.5	0.035	0.81	[0.05, 1.58]
Not Enough Rounds to Learn	-1.4	27.8	0.184	-0.49	[-1.24, 0.25]
# Correct Attributes Comparisons	0.0	29.0	0.974	0.01	[-0.72, 0.75]
Confidence in Attributes Comps.	2.8	26.1	0.009	1.00	[0.22, 1.78]
Extremity of Bets	3.6	28.4	0.001	1.28	[0.47, 2.09]

```

) %>%
tab_header(
  title = "Independent-Samples t-Tests: Increasing vs. Decreasing Condition"
)

```

3.3 Summary

Consistent with the results of Study 1, executives exposed to an improving sequence of outcomes reported a greater subjective understanding of cue-outcome relationships and greater confidence in their ability to generate positive outcomes in the future. Critically, the number of correct attribute comparisons—the only objective measure of learning—did not differ between conditions ($d = 0.01$, $p = .974$), confirming that the manipulation affected subjective beliefs about learning without improving actual knowledge of the cue-outcome relationships.

Appendix D - Replication of Study 1 with a Diagnostic Cue

We conducted a conceptual replication of Study 1 in which one of the cues reliably predicted participants' outcomes. This design tested whether exogenous trends can still foster illusory beliefs about non-diagnostic cues when participants have the opportunity to identify a genuinely predictive cue.

Participants played the same investment simulation as in Study 1, with two modifications. First, one of the three continuous company attributes (Company Valuation, Market Size, or Number of Competitors) was randomly selected to influence returns, while the other two continuous attributes and the categorical attribute (Sector of Activity) remained non-diagnostic. Second, we included only the Increasing and Decreasing conditions—the Flat condition was dropped to ensure a matched signal-to-noise ratio for detecting the diagnostic cue across conditions.

After 20 rounds, participants completed a post-game survey measuring (1) their subjective sense of learning—using the same three items as Study 1 ($r = .94$)—and (2) their endorsement of cue-outcome relationships involving the two non-diagnostic continuous attributes. Following the pre-registered interim analysis plan, data were collected in two waves of 400 participants each. The study was pre-registered on AsPredicted.org ([#db3p-fnz6](#)).

1 Preamble: Data Loadings and Wrangling

```
library(readr)
library(tidyr)
library(mlogit)
library(stringr)
library(glue)
library(gt)
library(gtsummary)
library(tidyverse)
library(tidymodels)
library(broom)
library(here)

i_am("Appendices/AppendixD.qmd")
```

```

source(here("custom_theme.R"))

generate_title <- function(model) {
  coeffs <- model %>% broom::tidy()
  summ <- model %>% broom::glance()
  df <- summ %>%
    dplyr::slice(1) %>%
    dplyr::pull(df.residual)
  t <- coeffs %>%
    dplyr::slice(2) %>%
    dplyr::pull(statistic)
  pval <- coeffs %>%
    dplyr::slice(2) %>%
    dplyr::pull(p.value)
  glue(
    "t({df}) = {round(t, 3)}, ",
    "{ifelse(pval < .001, 'p < .001',
    glue('p = {round(pval, 3)}'))}"
  )
}

# Load survey data
survey_data <- read_csv(
  here("Studies/Appendix_DiagCue", "Data", "survey_clean.csv")
)
game_data <- read_csv(
  here(
    "Studies/Appendix_DiagCue", "Data",
    "game_choices_clean.csv"
  )
)
runtime <- game_data %>%
  distinct(turkid, .keep_all = TRUE) %>%
  dplyr::select(c(turkid, time_to_completion))
survey_data_all <- survey_data %>%
  left_join(runtime, by = join_by(turkid)) %>%

```

```
dplyr::filter(time_to_completion > 90)
```

2 Sample

```
n_excluded <- nrow(survey_data) - nrow(survey_data_all)
n_total <- nrow(survey_data_all)
n_per <- survey_data_all %>% count(condition)
```

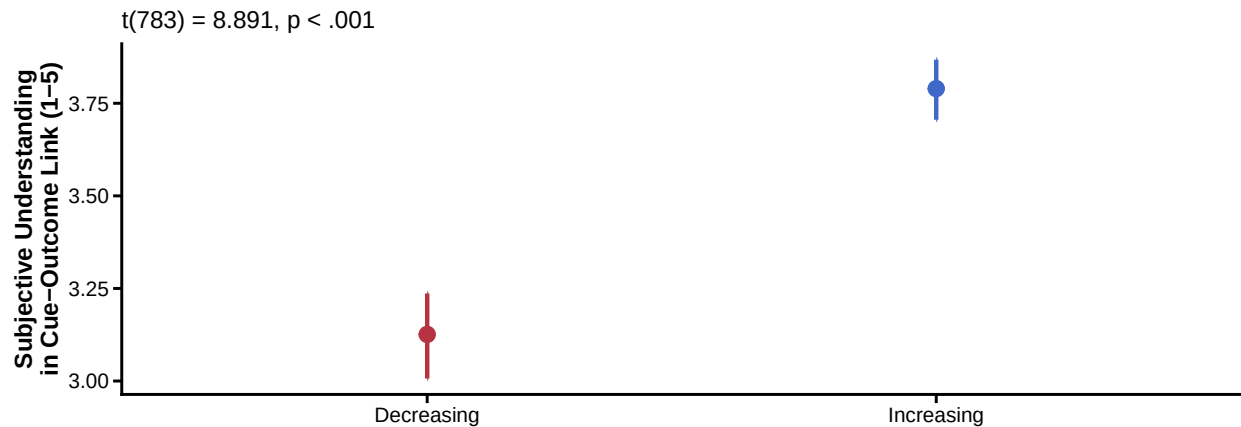
After excluding 15 participants who completed the cue-learning task in under 90 seconds, the final sample consisted of 785 participants (393 in the Increasing condition, 392 in the Decreasing condition).

3 Pre-Registered Dependent Variables

3.1 Subjective Learning

```
model <- lm(subjective_learning ~ condition, data = survey_data_all)
survey_data_all %>%
  ggplot(aes(
    x = factor(condition), y = subjective_learning,
    color = condition
  )) +
  stat_summary(
    fun = mean, geom = "point", alpha = 1,
    position = position_dodge(width = 0.5), size = 4
  ) +
  stat_summary(
    fun.data = mean_cl_boot, geom = "errorbar",
    width = 0, position = position_dodge(width = 0.5),
    show.legend = FALSE, linewidth = 1.2
  ) +
  labs(
    x = element_blank(),
    y = "Subjective Understanding\nin Cue-Outcome Link (1-5)",
    title = generate_title(model)
  ) +
```

```
theme_matplotlib() +
scale_color_manual(values = c("#b63441", "#3f69c9")) +
guides(color = "none")
```

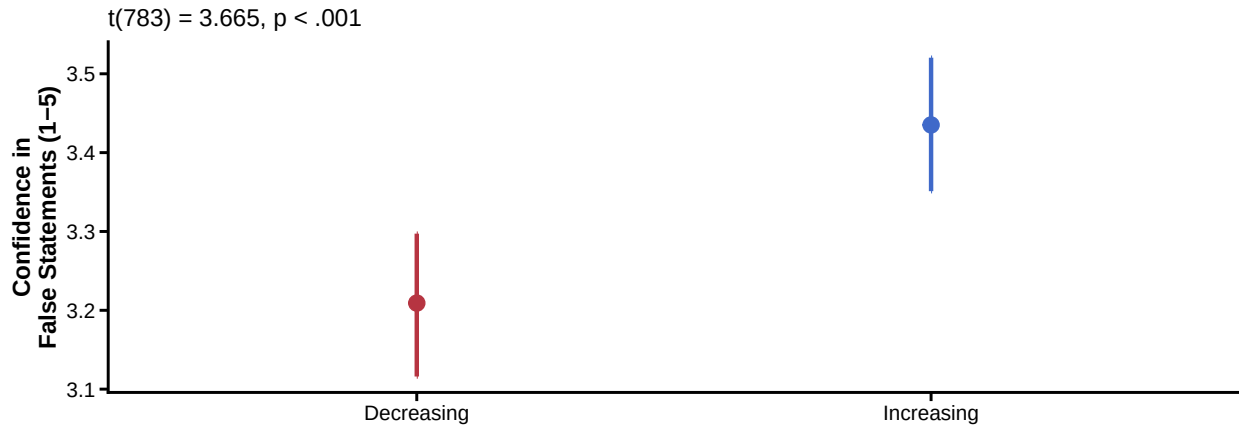


3.2 Confidence in False Statements

```
model <- lm(confidence ~ condition, data = survey_data_all)
survey_data_all %>%
  ggplot(aes(
    x = factor(condition), y = confidence,
    color = condition
  )) +
  stat_summary(
    fun = mean, geom = "point", alpha = 1,
    position = position_dodge(width = 0.5), size = 4
  ) +
  stat_summary(
    fun.data = mean_cl_boot, geom = "errorbar", width = 0,
    position = position_dodge(width = 0.5),
    show.legend = FALSE, linewidth = 1.2
  ) +
  labs(
    x = element_blank(),
    y = "Confidence in\nFalse Statements (1-5)",
    title = generate_title(model)
  ) +
```



```
theme_matplotlib() +
scale_color_manual(values = c("#b63441", "#3f69c9")) +
guides(color = "none")
```



4 Model: Cue by Condition Interactions

```
# Load the choice data for the multinomial logit model
choices_data <- read_csv(
  here(
    "Studies/Appendix_DiagCue", "Data",
    "game_choices_clean_with_attributes.csv"
  )
) %>%
  arrange(condition) %>%
  rename_with(
    ~ recode(
      .,
      "valuation" = "Valuation",
      "customers" = "Customers",
      "competitors" = "Competitors",
      "round" = "Round",
      "project" = "Project",
      "condition" = "Condition",
      "project_chosen" = "Project_Chosen"
    )
  )
```

```

    )

# Standardize Valuation, Customers, and Competitors
choices_data <- choices_data %>%
  mutate(
    Valuation = as.numeric(scale(Valuation)),
    Customers = as.numeric(scale(Customers)),
    Competitors = as.numeric(scale(Competitors))
  )

# Add dummy variables for activity and Condition
choices_data <- choices_data %>%
  mutate(
    Biotech = as.numeric(activity == "Biotechnology"),
    `Electronics` = as.numeric(activity == "Electronics"),
    `Robotics` = as.numeric(activity == "Robotics"),
    condid = condid / 2,
    Option_0 = as.numeric(Project == 0),
    Option_1 = as.numeric(Project == 1),
    Option_2 = as.numeric(Project == 2),
    Project_Chosen = as.numeric(Project_Chosen)
  ) %>%
  dplyr::arrange(turkid, Round, Project)

# Adjust Round and Slope
choices_data <- choices_data %>%
  mutate(
    Slope = condid,
    Round = Round - 1
  )

# Create interaction terms
interaction_columns <- c(
  "Option_1", "Option_2", "Valuation", "Customers", "Competitors",
  "Electronics", "Robotics"
)

```

```

for (c in interaction_columns) {
  choices_data <- choices_data %>%
    mutate(
      !!paste0("Slope_", c) := !!sym(c) * Slope,
      !!paste0("Round_", c) := !!sym(c) * Round,
      !!paste0("ThreeWay_", c) := !!sym(c) * Round * Slope
    )
}

# Sort the final data
choices_data_logit <- choices_data %>%
  mutate(choice_occasion = glue("{turkid}_{Round}")) %>%
  arrange(choice_occasion, Project)

# Restrict to choices made during the second half of the game
df_mxl <- mlogit.data(
  choices_data_logit %>% filter(Round > 9) %>% as.data.frame(),
  choice    = "Project_Chosen",
  shape     = "long",
  alt.var   = "Project",
  chid.var  = "choice_occasion", # task / round
  id.var    = "turkid" # respondent id (panel indicator)
)

```

4.1 Coefficients

```

# Interaction model: Cues + Cues:Condition interaction.
# Cache the full model object to avoid re-running the slow mlogit estimation.

cache_path <- here("Studies/Appendix_DiagCue", "Data", "mxl_int_model.rds")

if (file.exists(cache_path)) {
  mxl_int <- readRDS(cache_path)
} else {

```

```

mxl_int <- mlogit(
  Project_Chosen ~
    Customers + Competitors + Valuation +
    `Electronics` + `Robotics` +
    Slope_Customers + Slope_Competitors + Slope_Valuation +
    Slope_Electronics + Slope_Robotics |
    0,
  data = df_mxl,
  rpar = c(
    Customers = "n", Competitors = "n", Valuation = "n",
    `Electronics` = "n", `Robotics` = "n",
    Slope_Customers = "n", Slope_Competitors = "n",
    Slope_Valuation = "n",
    Slope_Electronics = "n",
    Slope_Robotics = "n"
  ),
  correlation = TRUE,
  panel = TRUE,
  R = 1000,
  halton = NA
)
saveRDS(mxl_int, cache_path)
}

model_inter_results <- mxl_int %>%
  tidy() %>%
  head(10) %>%
  mutate(
    term = c(
      "Customers", "Competitors", "Valuation",
      "Robotics vs. Biotech", "Electronics vs. Biotech",
      "Slope x Customers", "Slope x Competitors",
      "Slope x Valuation",
      "Slope x (Robo vs. Bio)",
      "Slope x (Elec vs. Bio)"
    )
  )

```

```

)
model_inter_results %>%
  gt(rowname_col = "term") %>%
  fmt_number(columns = c(estimate, std.error, statistic)) %>%
  fmt(columns = p.value, fns = style_pvalue) %>%
  tab_row_group(label = "Interaction Effects", rows = 6:10) %>%
  tab_row_group(label = "Main Effects", rows = 1:5) %>%
  cols_label(
    term = "Coeff.", estimate = "Estimate", std.error = "Std. Err.",
    statistic = "t-stats", p.value = "p-value"
  ) %>%
  as_raw_html()

```

	Estimate	Std. Err.	t-stats	p-value
Main Effects				
Customers	0.57	0.04	16.07	<0.001
Competitors	-0.75	0.03	-21.77	<0.001
Valuation	0.66	0.03	22.67	<0.001
Robotics vs. Biotech	-0.23	0.06	-4.20	<0.001
Electronics vs. Biotech	0.00	0.06	-0.05	>0.9
Interaction Effects				
Slope x Customers	0.08	0.03	2.36	0.018
Slope x Competitors	-0.21	0.03	-6.60	<0.001
Slope x Valuation	0.15	0.03	5.30	<0.001
Slope x (Robo vs. Bio)	-0.11	0.06	-1.99	0.046
Slope x (Elec vs. Bio)	-0.09	0.06	-1.64	0.10

4.2 Plotting the Marginal Effects

```

# Plotting the marginal effects:
## Define the contrasts matrix
contrasts <- do.call(rbind, lapply(c(-1, 1), function(i) {
  cbind(diag(5), diag(5) * i) # Combine diagonal matrices for each i
})))

```

```

## Define parameters
b <- mxl_int$coefficients[1:10] # Coefficients
vcv <- (-solve(mxl_int$hessian)[1:10, 1:10]) # Variance-Covariance Matrix

## Compute marginals
conts <- t(b %*% t(contrasts)) # Effects
errs <- sqrt(diag(contrasts %*% vcv %*% t(contrasts))) # Standard errors

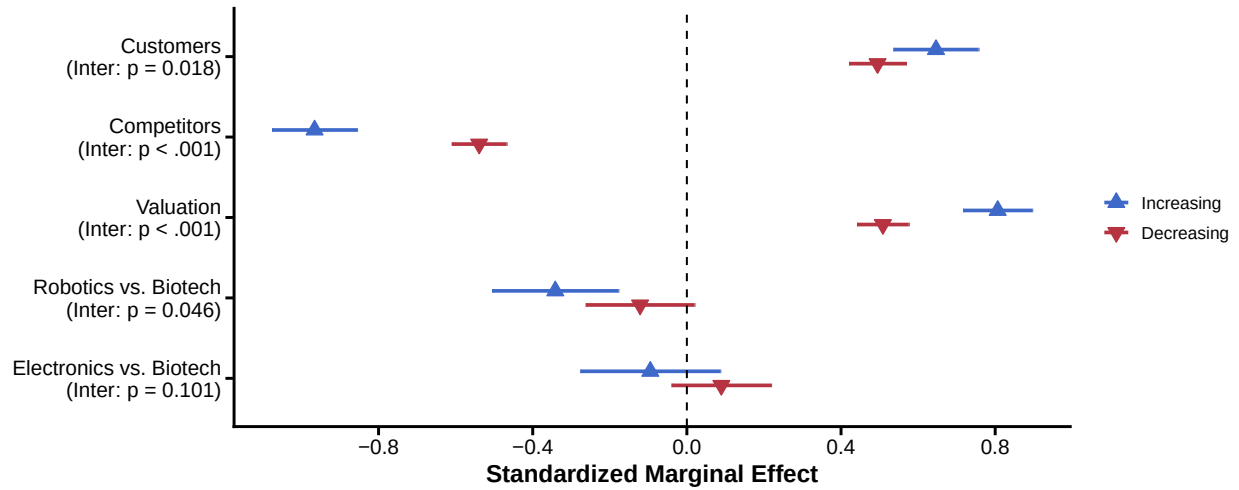
## Into a dataframe
names <- rep(
  c(
    "Customers", "Competitors", "Valuation",
    "Robotics vs. Biotech", "Electronics vs. Biotech"
  ),
  2
)
slope <- rep(c(-1, 1), each = 5)
levels <- c(
  "Customers", "Competitors", "Valuation",
  "Robotics vs. Biotech", "Electronics vs. Biotech"
)
pvals <- tail(ifelse(model_inter_results$p.value < .001, "p < .001",
  glue("p = {sprintf('%.3f', model_inter_results$p.value)}")
), 5)
labels <- glue("{levels}\n(Inter: {pvals})")
df_contrasts <- data.frame(
  estimate = conts, std.error = errs,
  term = factor(names,
    levels = levels,
    labels = labels
  ),
  slope = factor(slope,
    levels = c(1, -1),
    labels = c("Increasing", "Decreasing")
  )
)

```

```

## Plotting the results
ggplot(
  df_contrasts,
  aes(x = estimate, y = term, color = slope, shape = slope, fill = slope)
) +
  geom_point(size = 3, position = position_dodge(width = -.35)) +
  geom_errorbarh(
    aes(
      xmin = estimate - std.error * 1.96,
      xmax = estimate + std.error * 1.96
    ),
    height = 0, size = 1,
    position = position_dodge(width = -.35)
  ) +
  labs(
    title = element_blank(),
    x = "Standardized Marginal Effect",
    y = element_blank(),
    color = element_blank(),
    shape = element_blank(),
    fill = element_blank()
  ) +
  theme_matplotlib() +
  scale_x_continuous(breaks = seq(-16, 12, 4) / 10) +
  geom_vline(xintercept = 0, linetype = "dashed") +
  scale_y_discrete(limits = rev) +
  scale_color_manual(values = rev(c("#b63441", "#3f69c9"))) +
  scale_fill_manual(values = rev(c("#b63441", "#3f69c9"))) +
  scale_shape_manual(values = rev(c(25, 24)))

```



5 Summary

Replicating the pattern observed in Study 1, participants who experienced an increasing sequence of outcomes reported a significantly greater subjective sense of learning than those in the Decreasing condition. Critically, participants in the Increasing condition also expressed stronger endorsement of false statements about the non-diagnostic cues, confirming that exogenous trends can foster illusory beliefs even when one cue is genuinely predictive of outcomes. The analysis of participants' choices in the second half of the game further reveals that those in the Increasing condition attributed a greater weight to the cue-outcome relationships than those in the Decreasing condition—consistent with the behavioral signature of the positive test strategy documented in Study 1.

Appendix E - Analysis of Participants' Choices in Study 2

1 Preamble: Data Loadings and Wrangling

```
library(readr)
library(tidyr)
library(mlogit)
library(stringr)
library(glue)
library(gt)
library(gtsummary)
library(tidyverse)
library(tidymodels)
library(broom)
library(here)

i_am("Appendices/AppendixE.qmd")
source(here("custom_theme.R"))
```

2 Model: Cue by Condition Interactions

```
# Load the choice data for the multinomial logit model
choices_data <- read_csv(
  here(
    "Studies/Study2", "Data",
    "game_choices_clean_with_attributes.csv"
  )
) %>%
  arrange(condition) %>%
  rename_with(
    ~ recode(
      .,
      "valuation" = "Valuation",
```

```

        "customers" = "Customers",
        "competitors" = "Competitors",
        "round" = "Round",
        "project" = "Project",
        "condition" = "Condition",
        "project_chosen" = "Project_Chosen"
    )
)

# Standardize Valuation, Customers, and Competitors
choices_data <- choices_data %>%
  mutate(
    Valuation = as.numeric(scale(Valuation)),
    Customers = as.numeric(scale(Customers)),
    Competitors = as.numeric(scale(Competitors))
  )

# Add dummy variables for activity and Condition
choices_data <- choices_data %>%
  mutate(
    Biotech = as.numeric(activity == "Biotechnology"),
    `Electronics` = as.numeric(activity == "Electronics"),
    `Robotics` = as.numeric(activity == "Robotics"),
    Option_0 = as.numeric(Project == 0),
    Option_1 = as.numeric(Project == 1),
    Option_2 = as.numeric(Project == 2),
    Project_Chosen = as.numeric(Project_Chosen)
  ) %>%
  dplyr::arrange(turkid, Round, Project)

# Adjust Round and Slope
choices_data <- choices_data %>%
  mutate(
    Slope = condid,
    Round = Round - 1
  )

```

```

# Create interaction terms
interaction_columns <- c(
  "Option_1", "Option_2", "Valuation", "Customers", "Competitors",
  "Electronics", "Robotics"
)

for (c in interaction_columns) {
  choices_data <- choices_data %>%
    mutate(
      !!paste0("Slope_", c) := !!sym(c) * Slope,
      !!paste0("Round_", c) := !!sym(c) * Round,
      !!paste0("ThreeWay_", c) := !!sym(c) * Round * Slope
    )
}

# Sort the final data
choices_data_logit <- choices_data %>%
  mutate(choice_occasion = glue("{turkid}_{Round}")) %>%
  arrange(choice_occasion, Project)

# Restrict to choices made during the second half of the game
df_mxl <- mlogit.data(
  choices_data_logit %>% filter(Round > 9) %>% as.data.frame(),
  choice    = "Project_Chosen",
  shape     = "long",
  alt.var   = "Project",
  chid.var  = "choice_occasion", # task / round
  id.var    = "turkid" # respondent id (panel indicator)
)

```

2.1 Coefficients

```
# Interaction model: Cues + Cues:Condition interaction.
# Cache the full model object to avoid re-running the slow mlogit estimation.

cache_path <- here("Studies/Study2", "Data", "mxl_int_model.rds")

if (file.exists(cache_path)) {
  mxl_int <- readRDS(cache_path)
} else {
  mxl_int <- mlogit(
    Project_Chosen ~
      Customers + Competitors + Valuation +
      `Electronics` + `Robotics` +
      Slope_Customers + Slope_Competitors + Slope_Valuation +
      Slope_Electronics + Slope_Robotics |
      0,
    data = df_mxl,
    rpar = c(
      Customers = "n", Competitors = "n", Valuation = "n",
      `Electronics` = "n", `Robotics` = "n",
      Slope_Customers = "n", Slope_Competitors = "n",
      Slope_Valuation = "n",
      Slope_Electronics = "n",
      Slope_Robotics = "n"
    ),
    correlation = TRUE,
    panel = TRUE,
    R = 1000,
    halton = NA
  )
  saveRDS(mxl_int, cache_path)
}

model_inter_results <- mxl_int %>%
  tidy() %>%
```

```

head(10) %>%
mutate(
  term = c(
    "Customers", "Competitors", "Valuation",
    "Robotics vs. Biotech", "Electronics vs. Biotech",
    "Slope x Customers", "Slope x Competitors",
    "Slope x Valuation",
    "Slope x (Robo vs. Bio)",
    "Slope x (Elec vs. Bio)"
  )
)
model_inter_results %>%
  gt(rowname_col = "term") %>%
  fmt_number(columns = c(estimate, std.error, statistic)) %>%
  fmt(columns = p.value, fns = style_pvalue) %>%
  tab_row_group(label = "Interaction Effects", rows = 6:10) %>%
  tab_row_group(label = "Main Effects", rows = 1:5) %>%
  cols_label(
    term = "Coeff.", estimate = "Estimate", std.error = "Std. Err.",
    statistic = "t-stats", p.value = "p-value"
  ) %>%
  as_raw_html()

```

	Estimate	Std. Err.	t-stats	p-value
Main Effects				
Customers	0.35	0.05	6.93	<0.001
Competitors	-0.50	0.05	-10.38	<0.001
Valuation	0.36	0.05	7.89	<0.001
Robotics vs. Biotech	-0.16	0.10	-1.56	0.12
Electronics vs. Biotech	-0.07	0.09	-0.72	0.5
Interaction Effects				
Slope x Customers	0.18	0.09	2.01	0.044
Slope x Competitors	-0.47	0.09	-5.11	<0.001
Slope x Valuation	0.26	0.07	3.45	<0.001
Slope x (Robo vs. Bio)	-0.31	0.16	-1.89	0.059
Slope x (Elec vs. Bio)	0.04	0.16	0.23	0.8

2.2 Plotting the Marginal Effects

```
# Plotting the marginal effects:
## Define the contrasts matrix (Flat = 0, Increasing = 1)
contrasts <- do.call(rbind, lapply(c(0, 1), function(i) {
  cbind(diag(5), diag(5) * i) # Combine diagonal matrices for each i
}))

## Define parameters
b <- mxl_int$coefficients[1:10] # Coefficients
vcv <- (-solve(mxl_int$hessian)[1:10, 1:10]) # Variance-Covariance Matrix

## Compute marginals
conts <- t(b %*% t(contrasts)) # Effects
errs <- sqrt(diag(contrasts %*% vcv %*% t(contrasts))) # Standard errors

## Into a dataframe
names <- rep(
  c(
    "Customers", "Competitors", "Valuation",
    "Robotics vs. Biotech", "Electronics vs. Biotech"
  ),
  2
)
slope <- rep(c(0, 1), each = 5)
levels <- c(
  "Customers", "Competitors", "Valuation",
  "Robotics vs. Biotech", "Electronics vs. Biotech"
)
pvals <- tail(ifelse(model_inter_results$p.value < .001, "p < .001",
  glue("p = {sprintf('%.3f', model_inter_results$p.value)}")
), 5)
labels <- glue("{levels}\n(Inter: {pvals})")
df_contrasts <- data.frame(
  estimate = conts, std.error = errs,
  term = factor(names,
```

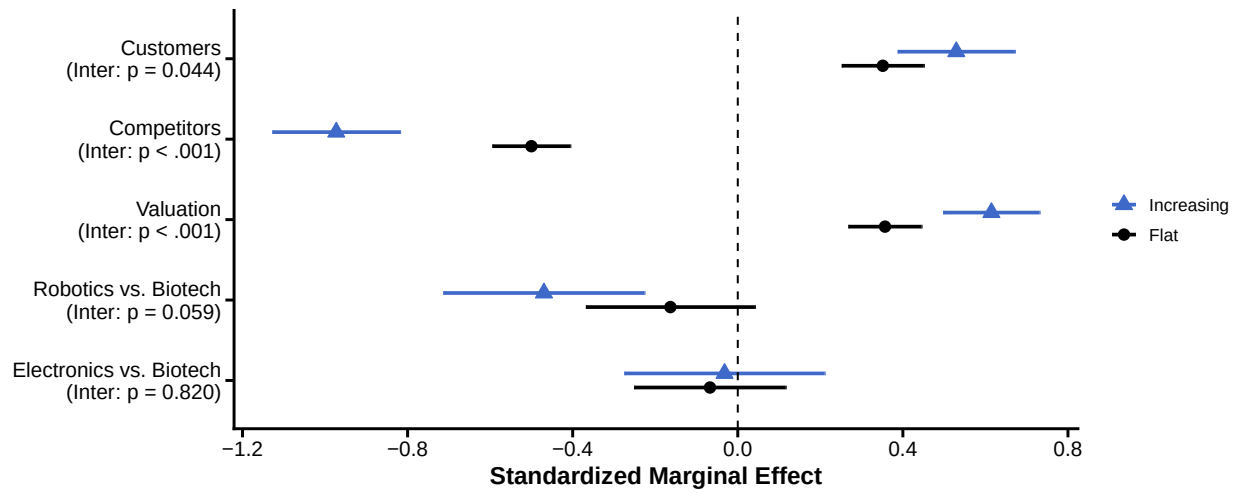
```

      levels = levels,
      labels = labels
    ),
    slope = factor(slope,
      levels = c(1, 0),
      labels = c("Increasing", "Flat")
    )
  )
)

## Plotting the results
ggplot(
  df_contrasts,
  aes(x = estimate, y = term, color = slope, shape = slope, fill = slope)
) +
  geom_point(size = 3, position = position_dodge(width = -.35)) +
  geom_errorbarh(
    aes(
      xmin = estimate - std.error * 1.96,
      xmax = estimate + std.error * 1.96
    ),
    height = 0, size = 1,
    position = position_dodge(width = -.35)
  ) +
  labs(
    title = element_blank(),
    x = "Standardized Marginal Effect",
    y = element_blank(),
    color = element_blank(),
    shape = element_blank(),
    fill = element_blank()
  ) +
  theme_matplotlib() +
  scale_x_continuous(breaks = seq(-16, 12, 4) / 10) +
  geom_vline(xintercept = 0, linetype = "dashed") +
  scale_y_discrete(limits = rev) +
  scale_color_manual(values = c("#3f69c9", "black")) +

```

```
scale_fill_manual(values = c("#3f69c9", "black")) +
scale_shape_manual(values = c(24, 16))
```



Appendix F - Full Results for Study 3b

1 Preamble: Data Loadings and Wrangling

```
library(tidyverse)
library(tidymodels)
library(glue)
library(lme4)
library(lmerTest)
library(forcats)
library(gt)
library(gtsummary)
library(broom) # For tidy and glance functions
library(broom.mixed)
library(dplyr) # For slice and pull functions
library(here)

i_am("Appendices/AppendixF.qmd")
source(here("custom_theme.R"))

# Declare global variables to suppress warnings
globalVariables(c("statistic", "p.value"))
## [1] "statistic" "p.value"

generate_title <- function(model) {
  coeffs <- model %>% tidy()
  summ <- model %>% glance()
  df <- summ %>% slice(1) %>% pull(df.residual)
  t <- coeffs %>% slice(2) %>% pull(statistic)
  pval <- coeffs %>% slice(2) %>% pull(p.value)
  glue(
    "t({df}) = {round(t, 3)}, {ifelse(pval < .001,
    'p < .001', glue('p = {round(pval, 3)}'))}"
  )
}
```

```

    )
  }

# Data loading
game_data <- read_csv(here("Studies/Study3b", "Data", "game_data_clean.csv")) %>%
  mutate(condition = factor(condition, levels = c("Flat", "Increasing")))

```

2 Analysis of Choices During the Game

2.1 Number of Attribute Changes

2.1.1 Descriptive Statistics

```

game_data %>%
  filter(week != 1) %>%
  group_by(condition, week) %>%
  summarise(
    mean = mean(n_changed)
  ) %>%
  ungroup() %>%
  pivot_wider(names_from=condition, values_from=mean) %>%
  gt() %>%
  fmt_number(columns = c(Flat, Increasing)) %>%
  tab_spanner(label = "Mean Number of Attribute Changes",
              columns=c(Flat, Increasing)) %>%
  cols_label(
    week = "Round", Flat = "Flat", Increasing="Increasing",
  ) %>%
  cols_align("center", columns=c(Flat, Increasing)) %>%
  as_raw_html()

```

Round	Mean Number of Attribute Changes	
	Flat	Increasing
2	1.97	1.99
3	1.93	1.92
4	1.89	1.70

Round	Mean Number of Attribute Changes	
	Flat	Increasing
5	2.07	2.08
6	2.06	1.69
7	2.00	1.43
8	1.64	1.16

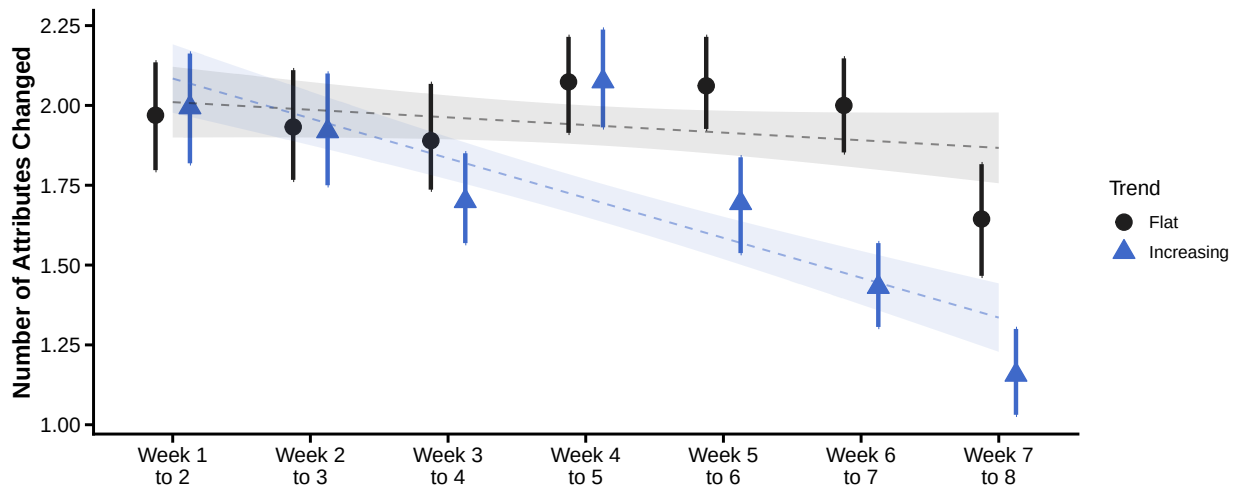
2.1.2 Graph

```
game_data %>%
  filter(week != 1) %>%
  ggplot(aes(
    x = week, y = n_changed, fill = condition,
    shape = condition, color = condition
  )) +
  stat_summary(
    fun = mean, geom = "point",
    position = position_dodge(width = 0.5), size = 4
  ) +
  stat_summary(
    fun.data = mean_cl_boot, geom = "errorbar", width = 0,
    position = position_dodge(width = 0.5), show.legend = FALSE, size = 1.2
  ) +
  geom_smooth(
    method = "lm", alpha = 0.1, linetype = 0, span = .5, show.legend = FALSE
  ) +
  geom_line(
    stat = "smooth", method = "lm", linetype = "dashed",
    alpha = .5, show.legend = FALSE
  ) +
  labs(
    y = "Number of Attributes Changed", x = element_blank(),
    color = "Trend", fill = "Trend", shape = "Trend"
  ) +
  scale_x_continuous(
```

```

breaks = seq(2, 8, 1),
labels = glue("Week {seq(1, 7)}\nto {seq(2, 8)}")
) +
theme_matplotlib() +
scale_color_manual(values = c("#1f1e1e", "#3f69c9")) +
scale_fill_manual(values = c("#1f1e1e", "#3f69c9")) +
scale_shape_manual(values = c(21, 24))

```



2.1.3 Model Results

```

lmer(
  formula = n_changed ~ week * condition + (1 + week | turkid),
  data = game_data %>% filter(week != 1)
) %>%
tidy() %>%
filter(effect == "fixed") %>%
dplyr::select(-c(effect, group)) %>%
mutate(term =
  c("Intercept", "Round", "Increasing", "Round x Increasing")
) %>%
gt(rowname_col = "term") %>%
fmt_number(columns = c(estimate, std.error, statistic, df)) %>%
fmt(columns = p.value, fns = style_pvalue) %>%
cols_label(
  estimate = "Estimate", std.error = "Std. Error", statistic="Statistic",

```

```
df = "DF"
) %>%
as_raw_html()
```

	Estimate	Std. Error	Statistic	DF	p.value
Intercept	2.06	0.10	21.09	321.01	<0.001
Round	-0.02	0.02	-1.59	321.01	0.11
Increasing	0.28	0.14	1.99	321.01	0.048
Round x Increasing	-0.10	0.02	-4.72	321.01	<0.001

2.2 Match to Best and Worst Practices

```
data_cue_matches <- game_data %>%
  dplyr::select(
    c(turkid, condition, week, n_matching_best, n_matching_worst)
  ) %>%
  pivot_longer(
    cols = c(n_matching_best, n_matching_worst),
    values_to = "n_matches"
  ) %>%
  mutate(
    feature_type = glue(
      '{str_to_title(gsub("n_matching_", "", name))} Practices'
    ) %>%
  )
  mutate(condition = factor(condition, levels = c("Flat", "Increasing")))
```

2.2.1 Descriptives

```
data_cue_matches %>%
  group_by(feature_type, week, condition) %>%
  summarise(
    mean = mean(n_matches)
  ) %>%
  ungroup() %>%
  pivot_wider(
    names_from=c(feature_type, condition),
```

```

    values_from=mean
  ) %>%
gt() %>%
fmt_number(
  columns = c(`Best Practices_Flat`, `Best Practices_Increasing`,
              `Worst Practices_Flat`, `Worst Practices_Increasing`)
  ) %>%
tab_spanner(label = "Cues Matching 'Best Practices'",
            columns=c(`Best Practices_Flat`, `Best Practices_Increasing`)
            ) %>%
tab_spanner(label = "Cues Matching 'Worst Practices'",
            columns=c(`Worst Practices_Flat`, `Worst Practices_Increasing`)
            ) %>%
cols_label(
  week = "Round", `Best Practices_Flat` = "Flat",
  `Worst Practices_Flat` = "Flat",
  `Best Practices_Increasing`="Increasing",
  `Worst Practices_Increasing`="Increasing",
) %>%
cols_align("center",
           columns=c(`Best Practices_Flat`,
                     `Worst Practices_Flat`,
                     `Best Practices_Increasing`,
                     `Worst Practices_Increasing`
                     )
           ) %>%
as_raw_html()

```

Round	Cues Matching 'Best Practices'		Cues Matching 'Worst Practices'	
	Flat	Increasing	Flat	Increasing
1	2.09	1.44	0.94	1.69
2	2.18	1.49	0.92	1.27
3	2.42	1.51	0.92	1.23
4	1.82	1.35	1.25	1.35
5	2.04	1.74	1.10	1.04
6	2.13	2.37	0.99	0.78

Round	Cues Matching 'Best Practices'		Cues Matching 'Worst Practices'	
	Flat	Increasing	Flat	Increasing
7	2.60	2.86	0.78	0.59
8	2.51	2.91	0.70	0.56

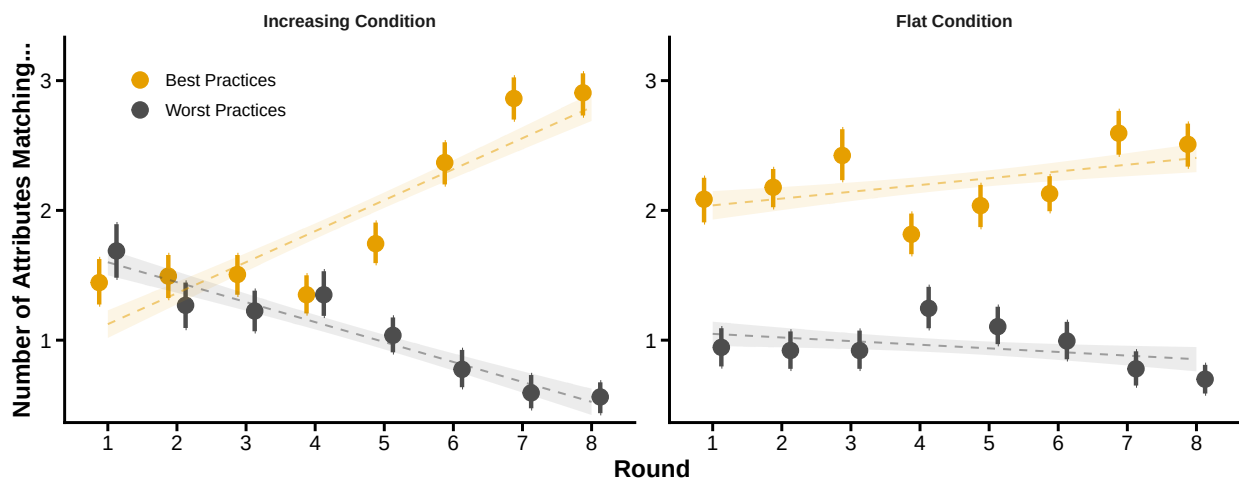
2.2.2 Graph

```
data_cue_matches %>%
  mutate(condition = fct_relevel(condition, "Increasing", "Flat")) %>%
  ggplot(aes(
    x = week, y = n_matches, color = feature_type, fill = feature_type
  )) +
  stat_summary(
    fun = mean, geom = "point",
    position = position_dodge(width = 0.5), size = 4
  ) +
  stat_summary(
    fun.data = mean_cl_boot, geom = "errorbar", width = 0,
    position = position_dodge(width = 0.5), show.legend = FALSE, size = 1.2
  ) +
  geom_smooth(
    method = "lm", alpha = 0.1, linetype = 0, span = .5, show.legend = FALSE
  ) +
  geom_line(
    stat = "smooth", method = "lm",
    linetype = "dashed", alpha = .5, show.legend = FALSE
  ) +
  scale_x_continuous(
    breaks = seq(1, 8, 1),
    labels = glue("{seq(1, 8)}")
  ) +
  theme_matplotlib() +
  facet_wrap(
    . ~ condition, scales = "free",
    labeller = as_labeller(function(x) glue("{x} Condition"))
  ) +
```

```

scale_shape_manual(values = c(21, 24)) +
labs(
  y = "Number of Attributes Matching...", x = "Round",
  color = element_blank(), fill = element_blank()
) +
theme(legend.position = c(.12, .85)) +
scale_fill_manual(
  values = c("Best Practices" = "#E69F00", "Worst Practices" = "#4D4D4D")
) +
scale_color_manual(
  values = c("Best Practices" = "#E69F00", "Worst Practices" = "#4D4D4D")
) +
coord_cartesian(ylim = c(0.5, 3.2))

```



2.2.3 Model for Match to 'Best Practices'

```

lmer(
  formula = n_matches ~ week * condition + (1 + week | turkid),
  data = data_cue_matches %>% filter(feature_type == "Best Practices")
) %>%
tidy() %>%
filter(effect == "fixed") %>%
dplyr::select(-c(effect, group)) %>%
mutate(term =

```



```

      c("Intercept", "Round", "Increasing", "Round x Increasing")
    ) %>%
gt(rownames_col = "term") %>%
fmt_number(columns = c(estimate, std.error, statistic, df)) %>%
fmt(columns = p.value, fns = style_pvalue) %>%
cols_label(
  estimate = "Estimate", std.error = "Std. Error", statistic="Statistic",
  df = "DF"
) %>%
as_raw_html()

```

	Estimate	Std. Error	Statistic	DF	p.value
Intercept	1.99	0.08	25.24	320.97	<0.001
Round	0.05	0.01	3.77	320.97	<0.001
Increasing	-1.10	0.11	-9.86	320.97	<0.001
Round x Increasing	0.19	0.02	9.47	320.97	<0.001

2.2.4 Model for Match to ‘Worst Practices’

```

lmer(
  formula = n_matches ~ week * condition + (1 + week | turkid),
  data = data_cue_matches %>% filter(feature_type == "Worst Practices")
) %>%
tidy() %>%
filter(effect == "fixed") %>%
dplyr::select(-c(effect, group)) %>%
mutate(term =
  c("Intercept", "Round", "Increasing", "Round x Increasing")
) %>%
gt(rownames_col = "term") %>%
fmt_number(columns = c(estimate, std.error, statistic, df)) %>%
fmt(columns = p.value, fns = style_pvalue) %>%
cols_label(
  estimate = "Estimate", std.error = "Std. Error", statistic="Statistic",
  df = "DF"
)

```

```
) %>%
as_raw_html()
```

	Estimate	Std. Error	Statistic	DF	p.value
Intercept	1.08	0.08	13.66	321.01	<0.001
Round	−0.03	0.01	−2.28	321.03	0.023
Increasing	0.68	0.11	6.05	321.01	<0.001
Round x Increasing	−0.13	0.02	−7.25	321.03	<0.001

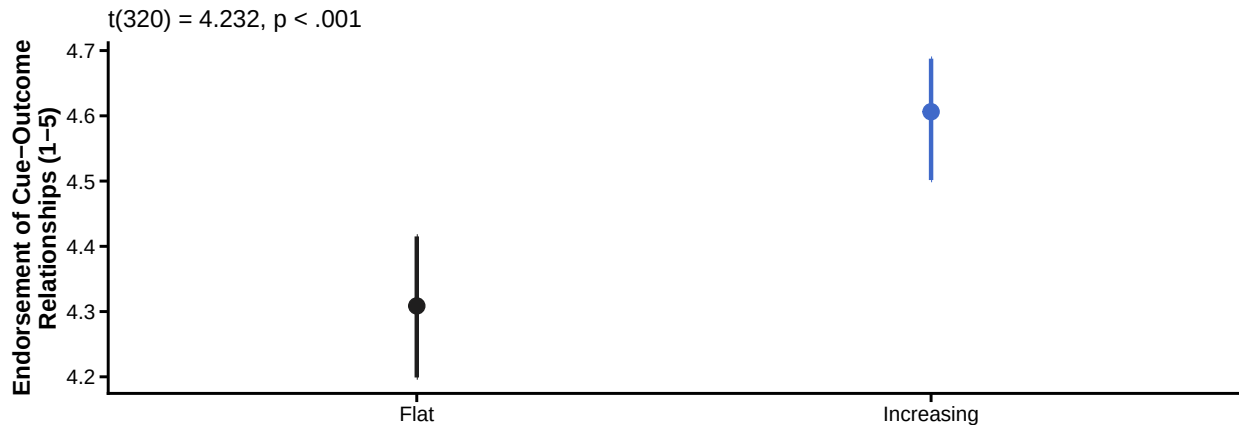
3 Survey Dependent Variables

```
survey_data_clean <- read_csv(here("Studies/Study3b", "Data", "survey_clean.csv")) %>%
  mutate(condition = factor(condition, levels = c("Flat", "Increasing")))
```

3.1 Endorsement of Cue-Outcome Relationships

```
model <- lm(illusory_beliefs_index ~ condition, data = survey_data_clean)
survey_data_clean %>%
  ggplot(aes(
    x = condition, y = illusory_beliefs_index,
    color = condition
  )) +
  stat_summary(
    fun = mean, geom = "point", alpha = 1,
    position = position_dodge(width = 0.5), size = 4
  ) +
  stat_summary(
    fun.data = mean_cl_boot, geom = "errorbar", width = 0,
    position = position_dodge(width = 0.5), show.legend = FALSE, linewidth = 1.2
  ) +
  labs(
    x = element_blank(),
    y = "Endorsement of Cue-Outcome
Relationships (1-5)",
    title = generate_title(model)
```

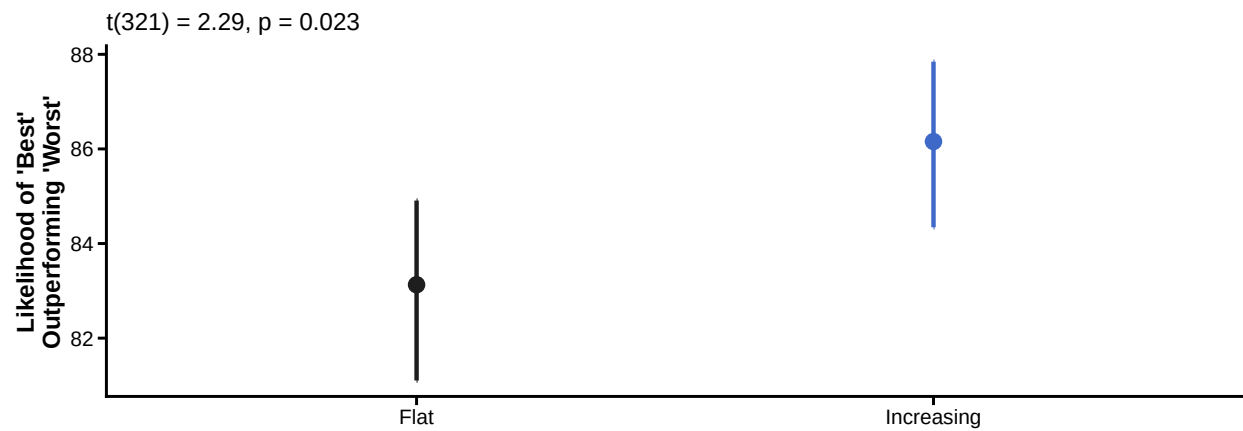
```
) +
theme_matplotlib() +
scale_color_manual(values = c("#1f1e1e", "#3f69c9")) +
guides(color = "none")
```



3.2 Probability of Superiority

```
model <- lm(prob_superiority ~ condition, data = survey_data_clean)
survey_data_clean %>%
  ggplot(aes(
    x = condition, y = prob_superiority,
    color = condition
  )) +
  stat_summary(
    fun = mean, geom = "point", alpha = 1,
    position = position_dodge(width = 0.5), size = 4
  ) +
  stat_summary(
    fun.data = mean_cl_boot, geom = "errorbar", width = 0,
    position = position_dodge(width = 0.5), show.legend = FALSE, linewidth = 1.2
  ) +
  labs(
    x = element_blank(),
    y = "Likelihood of 'Best'\nOutperforming 'Worst'",
    title = generate_title(model)
  ) +
```

```
theme_matplotlib() +  
scale_color_manual(values = c("#1f1e1e", "#3f69c9")) +  
guides(color = "none")
```



Appendix G - Full Results for Study 3c

1 Preamble: Data Loadings and Wrangling

```
library(tidyverse)
library(tidymodels)
library(glue)
library(lme4)
library(lmerTest)
library(forcats)
library(gt)
library(gtsummary)
library(broom) # For tidy and glance functions
library(broom.mixed)
library(dplyr) # For slice and pull functions
library(here)

i_am("Appendices/AppendixG.qmd")
source(here("custom_theme.R"))

# Declare global variables to suppress warnings
globalVariables(c("statistic", "p.value"))
## [1] "statistic" "p.value"

generate_title <- function(model) {
  coeffs <- model %>% tidy()
  summ <- model %>% glance()
  df <- summ %>% slice(1) %>% pull(df.residual)
  t <- coeffs %>% slice(2) %>% pull(statistic)
  pval <- coeffs %>% slice(2) %>% pull(p.value)
  glue(
    "t({df}) = {round(t, 3)}, {ifelse(pval < .001,
    'p < .001', glue('p = {round(pval, 3)}'))}"
  )
}
```

```

    )
  }

# Data loading
game_data <- read_csv(here("Studies/Study3c", "Data", "game_data_clean.csv")) %>%
  mutate(condition = factor(condition, levels = c("Flat", "Increasing")))

```

2 Analysis of Choices During the Game

2.1 Number of Attribute Changes

2.1.1 Descriptive Statistics

```

game_data %>%
  filter(day != 1) %>%
  group_by(condition, day) %>%
  summarise(
    mean = mean(n_changed)
  ) %>%
  ungroup() %>%
  pivot_wider(names_from=condition, values_from=mean) %>%
  gt() %>%
  fmt_number(columns = c(Flat, Increasing)) %>%
  tab_spanner(label = "Mean Number of Attribute Changes",
              columns=c(Flat, Increasing)) %>%
  cols_label(
    day = "Round", Flat = "Flat", Increasing="Increasing",
  ) %>%
  cols_align("center", columns=c(Flat, Increasing)) %>%
  as_raw_html()

```

Round	Mean Number of Attribute Changes	
	Flat	Increasing
2	1.59	1.62
3	1.62	1.57
4	1.62	1.48

Round	Mean Number of Attribute Changes	
	Flat	Increasing
5	1.81	1.82
6	1.79	1.57
7	1.68	1.36
8	1.47	1.13

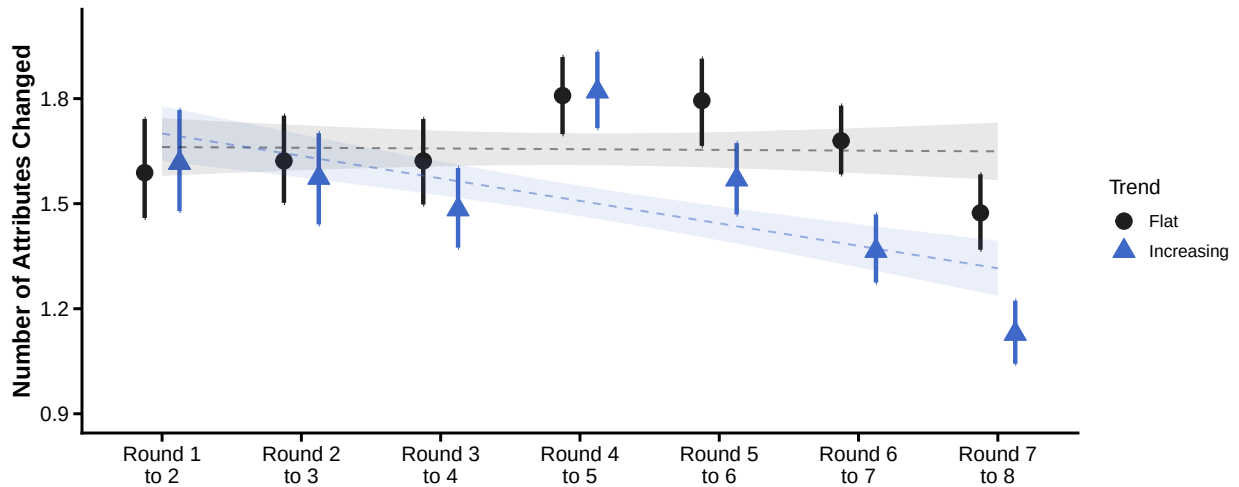
2.1.2 Graph

```
game_data %>%
  filter(day != 1) %>%
  ggplot(aes(
    x = day, y = n_changed, fill = condition,
    shape = condition, color = condition
  )) +
  stat_summary(
    fun = mean, geom = "point",
    position = position_dodge(width = 0.5), size = 4
  ) +
  stat_summary(
    fun.data = mean_cl_boot, geom = "errorbar", width = 0,
    position = position_dodge(width = 0.5), show.legend = FALSE, size = 1.2
  ) +
  geom_smooth(
    method = "lm", alpha = 0.1, linetype = 0, span = .5, show.legend = FALSE
  ) +
  geom_line(
    stat = "smooth", method = "lm", linetype = "dashed",
    alpha = .5, show.legend = FALSE
  ) +
  labs(
    y = "Number of Attributes Changed", x = element_blank(),
    color = "Trend", fill = "Trend", shape = "Trend"
  ) +
  scale_x_continuous(
```

```

breaks = seq(2, 8, 1),
labels = glue("Round {seq(1, 7)}\nto {seq(2, 8)}")
) +
coord_cartesian(ylim = c(0.9, 2)) +
theme_matplotlib() +
scale_color_manual(values = c("#1f1e1e", "#3f69c9")) +
scale_fill_manual(values = c("#1f1e1e", "#3f69c9")) +
scale_shape_manual(values = c(21, 24))

```



2.1.3 Model Results

```

lmer(
  formula = n_changed ~ day * condition + (1 + day | turkid),
  data = game_data %>% filter(day != 1)
) %>%
tidy() %>%
filter(effect == "fixed") %>%
dplyr::select(-c(effect, group)) %>%
mutate(term =
  c("Intercept", "Round", "Increasing", "Round x Increasing")
) %>%
gt(rowname_col = "term") %>%
fmt_number(columns = c(estimate, std.error, statistic, df)) %>%
fmt(columns = p.value, fns = style_pvalue) %>%
cols_label(

```



```

estimate = "Estimate", std.error = "Std. Error", statistic="Statistic",
df = "DF"
) %>%
as_raw_html()

```

	Estimate	Std. Error	Statistic	DF	p.value
Intercept	1.67	0.08	20.12	418.00	<0.001
Round	0.00	0.01	-0.18	418.00	0.9
Increasing	0.16	0.12	1.39	418.00	0.2
Round x Increasing	-0.06	0.02	-3.76	418.00	<0.001

2.2 Match to Best and Worst Practices

```

data_cue_matches <- game_data %>%
  dplyr::select(
    c(turkid, condition, day, n_matching_best, n_matching_worst)
  ) %>%
  pivot_longer(
    cols = c(n_matching_best, n_matching_worst),
    values_to = "n_matches"
  ) %>%
  mutate(
    feature_type = glue(
      '{str_to_title(gsub("n_matching_", "", name))} Practices'
    ) %>%
    mutate(condition = factor(condition, levels = c("Flat", "Increasing")))
  )

```

2.2.1 Descriptives

```

data_cue_matches %>%
  group_by(feature_type, day, condition) %>%
  summarise(
    mean = mean(n_matches)
  ) %>%
  ungroup() %>%
  pivot_wider(

```

```

names_from=c(feature_type, condition),
values_from=mean
) %>%
gt() %>%
fmt_number(
  columns = c(`Best Practices_Flat`, `Best Practices_Increasing`,
              `Worst Practices_Flat`, `Worst Practices_Increasing`)
) %>%
tab_spanner(label = "Cues Matching 'Best Practices'",
            columns=c(`Best Practices_Flat`, `Best Practices_Increasing`)
) %>%
tab_spanner(label = "Cues Matching 'Worst Practices'",
            columns=c(`Worst Practices_Flat`, `Worst Practices_Increasing`)
) %>%
cols_label(
  day = "Round", `Best Practices_Flat` = "Flat",
  `Worst Practices_Flat` = "Flat",
  `Best Practices_Increasing`="Increasing",
  `Worst Practices_Increasing`="Increasing",
) %>%
cols_align("center",
  columns=c(`Best Practices_Flat`,
            `Worst Practices_Flat`,
            `Best Practices_Increasing`,
            `Worst Practices_Increasing`
  )
) %>%
as_raw_html()

```

Round	Cues Matching 'Best Practices'		Cues Matching 'Worst Practices'	
	Flat	Increasing	Flat	Increasing
1	1.95	1.01	1.19	2.39
2	2.09	1.30	0.99	1.87
3	2.51	1.55	1.06	1.64
4	1.97	1.26	1.40	1.82
5	2.07	1.70	1.31	1.45

Round	Cues Matching 'Best Practices'		Cues Matching 'Worst Practices'	
	Flat	Increasing	Flat	Increasing
6	2.12	2.28	1.23	0.97
7	2.56	2.83	1.01	0.81
8	2.33	2.95	0.97	0.63

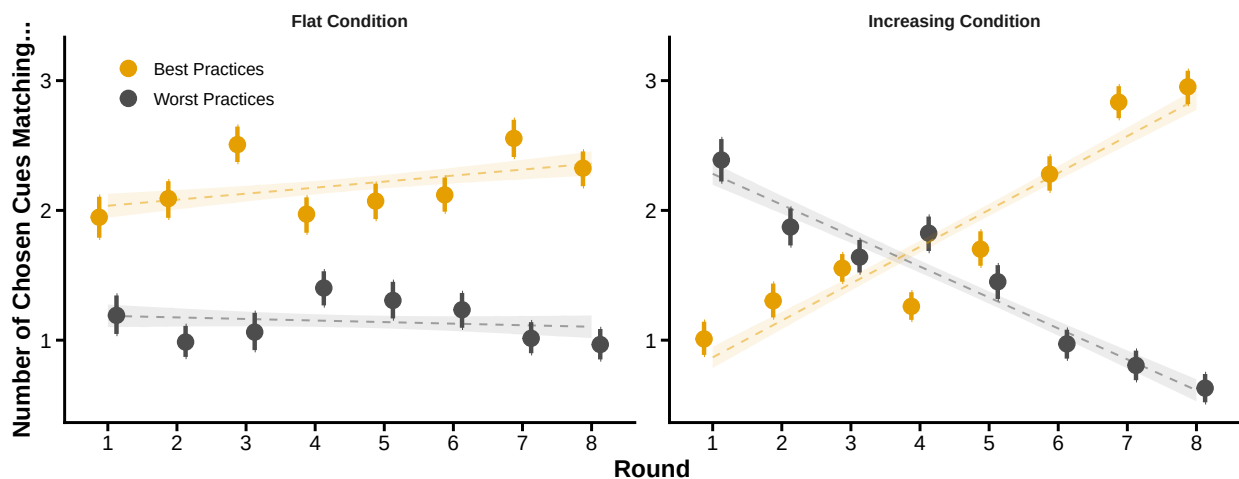
2.2.2 Graph

```
data_cue_matches %>%
  ggplot(aes(
    x = day, y = n_matches, color = feature_type, fill = feature_type
  )) +
  stat_summary(
    fun = mean, geom = "point",
    position = position_dodge(width = 0.5), size = 4
  ) +
  stat_summary(
    fun.data = mean_cl_boot, geom = "errorbar", width = 0,
    position = position_dodge(width = 0.5), show.legend = FALSE, size = 1.2
  ) +
  geom_smooth(
    method = "lm", alpha = 0.1, linetype = 0, span = .5, show.legend = FALSE
  ) +
  geom_line(
    stat = "smooth", method = "lm",
    linetype = "dashed", alpha = .5, show.legend = FALSE
  ) +
  scale_x_continuous(
    breaks = seq(1, 8, 1),
    labels = glue("{seq(1, 8)}")
  ) +
  theme_matplotlib() +
  facet_wrap(
    . ~ condition, scales = "free",
    labeller = as_labeller(function(x) glue("{x} Condition"))
  ) +
```

```

scale_shape_manual(values = c(21, 24)) +
labs(
  y = "Number of Chosen Cues Matching...", x = "Round",
  color = element_blank(), fill = element_blank()
) +
theme(legend.position = c(.11, .88)) +
scale_fill_manual(
  values = c("Best Practices" = "#E69F00", "Worst Practices" = "#4D4D4D")
) +
scale_color_manual(
  values = c("Best Practices" = "#E69F00", "Worst Practices" = "#4D4D4D")
) +
coord_cartesian(ylim = c(.5, 3.2))

```



2.2.3 Model for Match to 'Best Practices'

```

lmer(
  formula = n_matches ~ day * condition + (1 + day | turkid),
  data = data_cue_matches %>% filter(feature_type == "Best Practices")
) %>%
tidy() %>%
filter(effect == "fixed") %>%
dplyr::select(-c(effect, group)) %>%
mutate(term =

```

```

      c("Intercept", "Round", "Increasing", "Round x Increasing")
    ) %>%
gt(rownames_col = "term") %>%
fmt_number(columns = c(estimate, std.error, statistic, df)) %>%
fmt(columns = p.value, fns = style_pvalue) %>%
cols_label(
  estimate = "Estimate", std.error = "Std. Error", statistic="Statistic",
  df = "DF"
) %>%
as_raw_html()

```

	Estimate	Std. Error	Statistic	DF	p.value
Intercept	1.99	0.07	29.16	418.00	<0.001
Round	0.05	0.01	3.84	418.00	<0.001
Increasing	-1.41	0.10	-14.61	418.00	<0.001
Round x Increasing	0.24	0.02	13.91	418.00	<0.001

2.2.4 Model for Match to ‘Worst Practices’

```

lmer(
  formula = n_matches ~ day * condition + (1 + day | turkid),
  data = data_cue_matches %>% filter(feature_type == "Worst Practices")
) %>%
tidy() %>%
filter(effect == "fixed") %>%
dplyr::select(-c(effect, group)) %>%
mutate(term =
  c("Intercept", "Round", "Increasing", "Round x Increasing")
) %>%
gt(rownames_col = "term") %>%
fmt_number(columns = c(estimate, std.error, statistic, df)) %>%
fmt(columns = p.value, fns = style_pvalue) %>%
cols_label(
  estimate = "Estimate", std.error = "Std. Error", statistic="Statistic",
  df = "DF"
)

```

```
) %>%
as_raw_html()
```

	Estimate	Std. Error	Statistic	DF	p.value
Intercept	1.20	0.07	16.94	417.97	<0.001
Round	-0.01	0.01	-0.99	417.98	0.3
Increasing	1.32	0.10	13.22	417.97	<0.001
Round x Increasing	-0.23	0.02	-13.21	417.98	<0.001

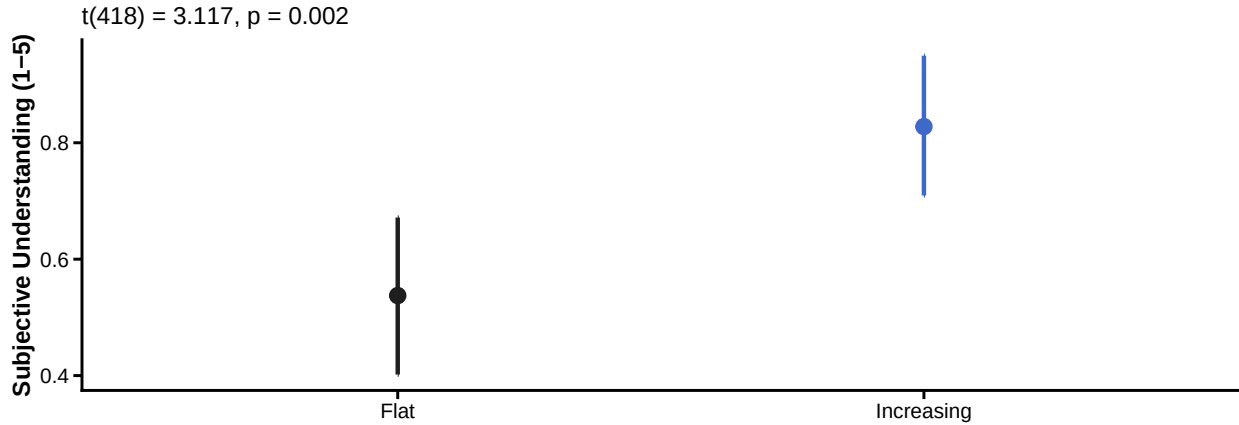
3 Survey Dependent Variables

```
survey_data_clean <- read_csv(here("Studies/Study3c", "Data", "survey_clean.csv")) %>%
  mutate(condition = factor(condition, levels = c("Flat", "Increasing")))
```

3.1 Confidence in Learning

```
model <- lm(confidence_in_learning_index ~ condition, data = survey_data_clean)
survey_data_clean %>%
  ggplot(aes(
    x = condition, y = confidence_in_learning_index,
    color = condition
  )) +
  stat_summary(
    fun = mean, geom = "point", alpha = 1,
    position = position_dodge(width = 0.5), size = 4
  ) +
  stat_summary(
    fun.data = mean_cl_boot, geom = "errorbar", width = 0,
    position = position_dodge(width = 0.5), show.legend = FALSE, linewidth = 1.2
  ) +
  labs(
    x = element_blank(),
    y = "Subjective Understanding (1-5)",
    title = generate_title(model)
  ) +
```

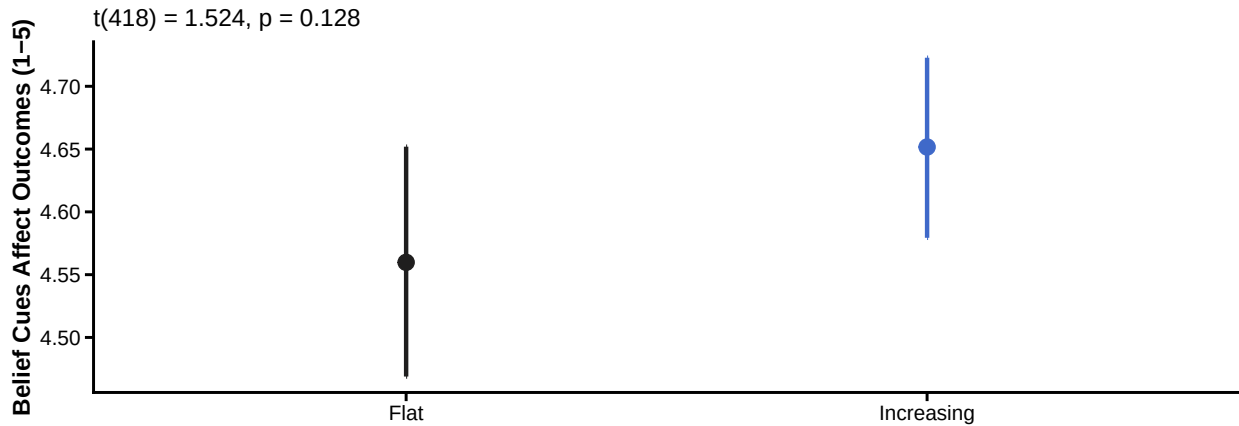
```
theme_matplotlib() +
scale_color_manual(values = c("#1f1e1e", "#3f69c9")) +
guides(color = "none")
```



3.2 Endorsement of Cue-Outcome Relationships

```
model <- lm(illusory_beliefs_index ~ condition, data = survey_data_clean)
survey_data_clean %>%
  ggplot(aes(
    x = condition, y = illusory_beliefs_index,
    color = condition
  )) +
  stat_summary(
    fun = mean, geom = "point", alpha = 1,
    position = position_dodge(width = 0.5), size = 4
  ) +
  stat_summary(
    fun.data = mean_cl_boot, geom = "errorbar", width = 0,
    position = position_dodge(width = 0.5), show.legend = FALSE, linewidth = 1.2
  ) +
  labs(
    x = element_blank(),
    y = "Belief Cues Affect Outcomes (1-5)",
    title = generate_title(model)
  ) +
  theme_matplotlib() +
```

```
scale_color_manual(values = c("#1f1e1e", "#3f69c9")) +
guides(color = "none")
```

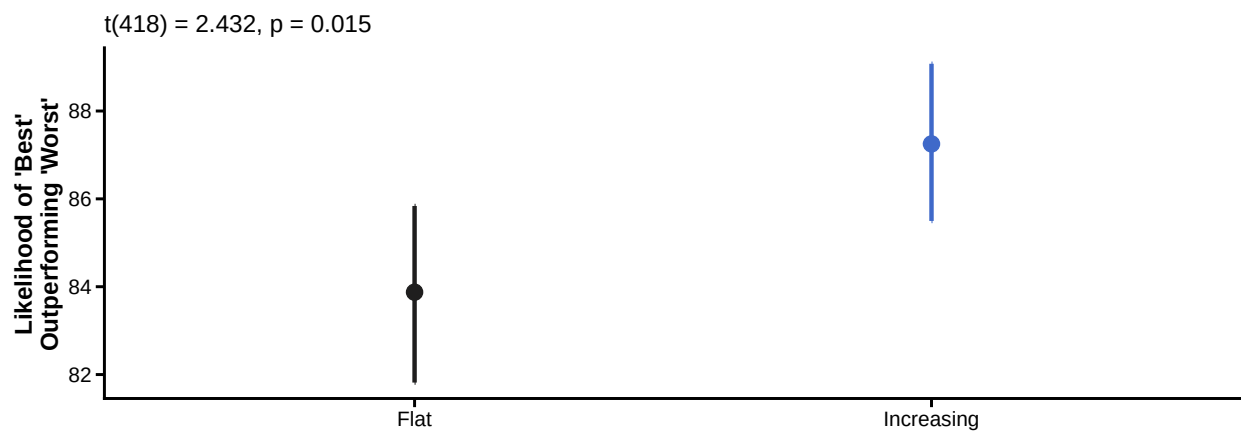


3.3 Probability of Superiority

```
model <- lm(prob_superiority ~ condition, data = survey_data_clean)
survey_data_clean %>%
  ggplot(aes(
    x = condition, y = prob_superiority,
    color = condition
  )) +
  stat_summary(
    fun = mean, geom = "point", alpha = 1,
    position = position_dodge(width = 0.5), size = 4
  ) +
  stat_summary(
    fun.data = mean_cl_boot, geom = "errorbar", width = 0,
    position = position_dodge(width = 0.5), show.legend = FALSE, linewidth = 1.2
  ) +
  labs(
    x = element_blank(),
    y = "Likelihood of 'Best'\nOutperforming 'Worst'",
    title = generate_title(model)
  ) +
  theme_matplotlib() +
  scale_color_manual(values = c("#1f1e1e", "#3f69c9")) +
```



```
guides(color = "none")
```



Appendix H - Analysis of Testing Strategies in Study 4

1 Preamble: Data Loading and Wrangling

```
library(tidyverse)
library(tidymodels)
library(glue)
library(gt)
library(mlogit)
library(here)

i_am("Appendices/AppendixH.qmd")
source(here("custom_theme.R"))

cols_subj_learn_interim <- c(
  "could_learn_interim_1",
  "could_learn_interim_2",
  "could_learn_interim_3"
)

cols_illusory_beliefs_interim <- c(
  "belief_valuation",
  "belief_market_size",
  "belief_sector_of_activity",
  "belief_competitors"
)

survey_data <- read_csv(here("Studies/Study4", "Data", "survey_clean.csv")) %>%
  mutate(Slope = 2 - Slope) %>%
  rowwise() %>%
  mutate(
    SubjectiveLearning_Interim = mean(c_across(cols_subj_learn_interim)),
    IllusoryBeliefs_Interim = mean(c_across(cols_illusory_beliefs_interim)),
```

```

    condition = factor(
      case_when(
        condid == 0 ~ "Flat",
        condid == 1 ~ "Increasing"
      ),
      levels = c("Flat", "Increasing")
    ),
    message_type = factor(
      message_type,
      levels = c("control", "positive_test", "negative_test"),
      labels = c("Control", "Positive\nTest", "Negative\nTest")
    )
  ) %>%
  dplyr::filter(time_to_completion > 110)

```

2 Analysis of Testing Strategies

Participants were asked to report the extent to which they used a positive testing strategy (seeking confirming evidence) and a negative testing strategy (seeking disconfirming evidence) in the first half (Rounds 1–10) and second half (Rounds 11–20) of the game.

```

testing_strategy <- survey_data %>%
  dplyr::select(
    turkid,
    condid,
    message_type,
    Positive_Test_First,
    Positive_Test_Second,
    Negative_Test_First,
    Negative_Test_Second
  ) %>%
  pivot_longer(
    cols = c(
      Positive_Test_First,
      Positive_Test_Second,
      Negative_Test_First,

```

```

        Negative_Test_Second
      )
    ) %>%
    mutate(
      Strategy = str_split(name, "_") %>% map_chr(~ .x[[1]]),
      Timing = str_split(name, "_") %>% map_chr(~ .x[[3]])
    )

label_map_time <- c("First" = "Rounds 1-10", "Second" = "Rounds 11-20")
label_strategy <- c(
  "Positive" = "... a Positive Test Strategy",
  "Negative" = "... a Negative Test Strategy"
)

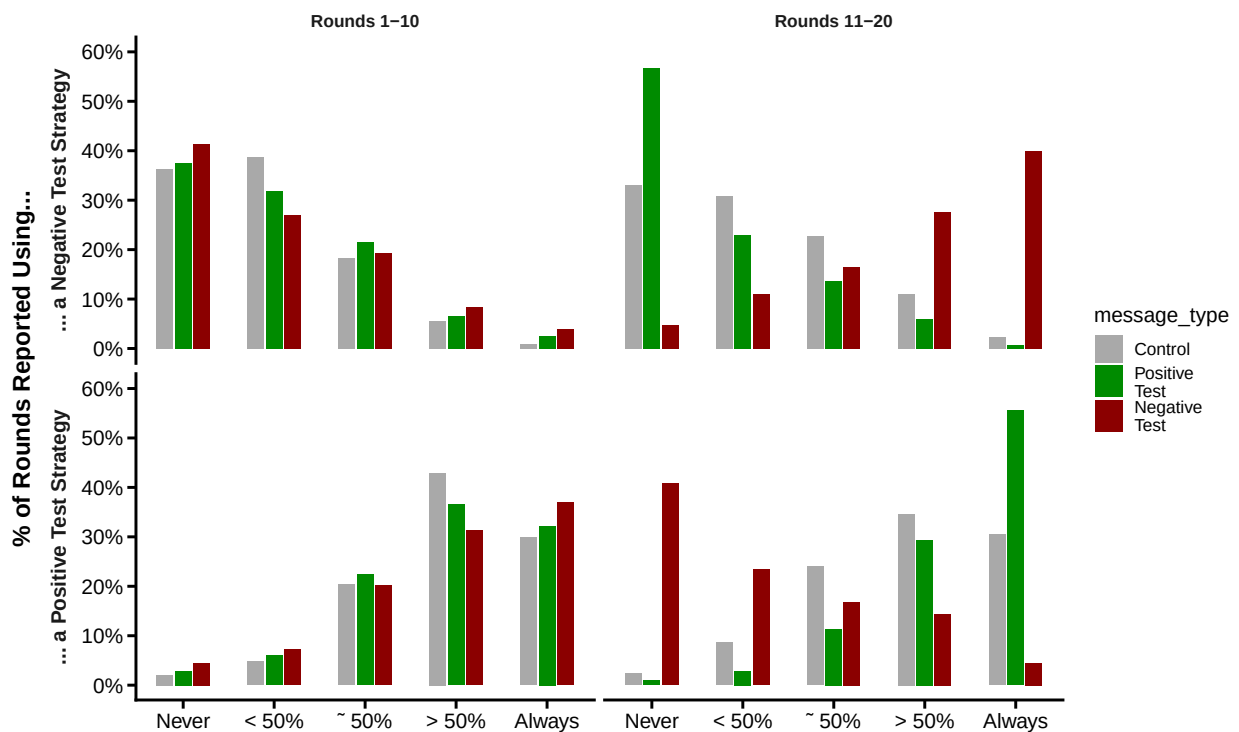
testing_strategy %>%
  group_by(message_type, Timing, Strategy) %>%
  count(value) %>%
  mutate(pct = n / sum(n)) %>%
  ggplot(aes(x = value, y = pct, fill = message_type)) +
  geom_col(position = position_dodge2(), width = .6) +
  facet_grid(
    Strategy ~ Timing,
    switch = "y",
    labeller = labeller(Timing = label_map_time, Strategy = label_strategy)
  ) +
  scale_fill_manual(values = c("darkgrey", "green4", "red4")) +
  scale_x_continuous(
    breaks = c(1, 2, 3, 4, 5),
    labels = c("Never", "< 50%", "\u2248 50%", "> 50%", "Always")
  ) +
  labs(
    x = element_blank(),
    y = "% of Rounds Reported Using...",
    color = element_blank()
  ) +
  scale_y_continuous(

```

```

limits = c(0, .60),
breaks = seq(0, .60, by = .10),
labels = scales::label_percent()
) +
theme_matplotlib() +
theme(
  strip.placement = "outside",
  strip.text.y.left = element_text(angle = 90, vjust = 0.5, size = 12)
)

```



The left panels (Rounds 1–10) show that participants rarely used a Negative Test strategy (top panel) and instead favored a Positive Test strategy (bottom panel).

The right panels (Rounds 11–20) confirm the effectiveness of our testing prompt: Participants in the “Negative Test” condition substantially increased their use of a Negative Test strategy (top right panel) and decreased their use of a Positive Test strategy (bottom right panel). In contrast, participants in the “Positive Test” condition further increased their use of a Positive Test strategy while maintaining a low use of a Negative Test strategy.

3 Further Evidence of Change in Testing Strategies

A multinomial logit model (estimated on the choices made in Rounds 15–20, after the testing prompt was presented) provides further evidence that the prompt changed participants' testing strategies.

```
# Load and prepare game choices data for the multinomial logit model
choices_data <- read_csv(
  here(
    "Studies/Study4", "Data",
    "game_choices_clean_with_attributes.csv"
  )
) %>%
  arrange(condition) %>%
  rename_with(
    ~ recode(
      .,
      "valuation" = "Valuation",
      "customers" = "Customers",
      "competitors" = "Competitors",
      "round" = "Round",
      "project" = "Project",
      "condition" = "Condition",
      "project_chosen" = "Project_Chosen"
    )
  ) %>%
  mutate(
    Valuation = as.numeric(scale(Valuation)),
    Customers = as.numeric(scale(Customers)),
    Competitors = as.numeric(scale(Competitors)),
    `Biotech` = as.numeric(activity == "Biotechnology"),
    `Electronics` = as.numeric(activity == "Electronics"),
    `Robotics` = as.numeric(activity == "Robotics"),
    Flat = as.numeric(Condition == "Flat"),
    Inc = as.numeric(Condition == "Increasing"),
    Pos = as.numeric(message_type == "positive_test"),
```

```

    Neg = as.numeric(message_type == "negative_test"),
    Option_0 = as.numeric(Project == 0),
    Option_1 = as.numeric(Project == 1),
    Option_2 = as.numeric(Project == 2),
    Project_Chosen = as.numeric(Project_Chosen),
    Slope = condid,
    Round = Round - 1
  ) %>%
  dplyr::arrange(turkid, Round, Project)

interaction_columns <- c(
  "Option_1", "Option_2", "Valuation", "Customers", "Competitors",
  "Electronics", "Robotics"
)

for (col in interaction_columns) {
  for (m in c("Pos", "Neg")) {
    choices_data <- choices_data %>%
      mutate(!paste0(m, "_", col) := !!sym(col) * !!sym(m))
  }
  choices_data <- choices_data %>%
    mutate(
      !!paste0("Slope_", col) := !!sym(col) * Slope,
      !!paste0("Round_", col) := !!sym(col) * Round,
      !!paste0("ThreeWay_", col) := !!sym(col) * Round * Slope
    )
}

choices_data_logit <- choices_data %>%
  mutate(
    choice_occasion = glue("{turkid}_{Round}"),
    turkid = as.integer(factor(turkid))
  ) %>%
  arrange(turkid, Round, Project)

df_mxl <- mlogit.data(

```

```

choices_data_logit %>%
  filter(Round > 14 & Inc == 1) %>%
  as.data.frame() |>
  dplyr::select(
    turkid, choice_occasion, Project, Project_Chosen,
    Customers, Competitors, Valuation, Electronics, Robotics,
    Pos_Customers, Pos_Competitors, Pos_Valuation,
    Pos_Electronics, Pos_Robotics,
    Neg_Customers, Neg_Competitors, Neg_Valuation,
    Neg_Electronics, Neg_Robotics
  ),
  choice = "Project_Chosen",
  shape = "long",
  alt.var = "Project",
  chid.var = "choice_occasion",
  id.var = "turkid"
)

# Estimate the mixed logit model (cached to avoid re-running)
cache_path <- here("Studies/Study4", "Data", "mxl_int_model.rds")

if (file.exists(cache_path)) {
  mxl_int <- readRDS(cache_path)
} else {
  mxl_int <- mlogit(
    Project_Chosen ~
      Customers + Competitors + Valuation +
      Electronics + Robotics +
      Pos_Customers + Pos_Competitors + Pos_Valuation +
      Pos_Electronics + Pos_Robotics +
      Neg_Customers + Neg_Competitors + Neg_Valuation +
      Neg_Electronics + Neg_Robotics |
      0,
    data = df_mxl,
    rpar = c(
      Customers = "n", Competitors = "n", Valuation = "n",

```



```

        Electronics = "n", Robotics = "n",
        Pos_Customers = "n", Pos_Competitors = "n", Pos_Valuation = "n",
        Pos_Electronics = "n", Pos_Robotics = "n",
        Neg_Customers = "n", Neg_Competitors = "n", Neg_Valuation = "n",
        Neg_Electronics = "n", Neg_Robotics = "n"
    ),
    correlation = TRUE,
    panel = TRUE,
    R = 1000,
    halton = NA
)
saveRDS(mx1_int, cache_path)
}

# Compute marginal effects via contrasts
model_inter_results <- mx1_int %>%
  tidy() %>%
  head(15)

b <- mx1_int$coefficients[1:15]
vcv <- (-solve(mx1_int$hessian)[1:15, 1:15])

k <- 5
contrasts_mat <- rbind(
  cbind(diag(k), 0 * diag(k), 0 * diag(k)), # Control
  cbind(diag(k), diag(k), 0 * diag(k)), # Pos Test
  cbind(diag(k), 0 * diag(k), diag(k)) # Neg Test
)

conts <- as.numeric(contrasts_mat %*% b)
errs <- sqrt(diag(contrasts_mat %*% vcv %*% t(contrasts_mat)))

levels <- c(
  "Customers", "Competitors", "Valuation",
  "Electronics vs. Biotech", "Robotics vs. Biotech"
)

```

```

names <- rep(levels, times = 3)

pvals_pos <- ifelse(model_inter_results$p.value < .001, "p < .001",
  glue("p = {sprintf('%.3f', model_inter_results$p.value)}")
)[6:10]
pvals_neg <- ifelse(model_inter_results$p.value < .001, "p < .001",
  glue("p = {sprintf('%.3f', model_inter_results$p.value)}")
)[11:15]
labels <- glue("{levels}\n(Inter_Pos: {pvals_pos})\n(Inter_Neg: {pvals_neg})")

df_contrasts <- data.frame(
  estimate = conts,
  std.error = errs,
  term = names,
  message_type = factor(
    rep(c("Control", "Positive Test", "Negative Test"), each = k),
    levels = c("Control", "Positive Test", "Negative Test")
  ),
  term_names = factor(names, levels = levels, labels = labels)
)

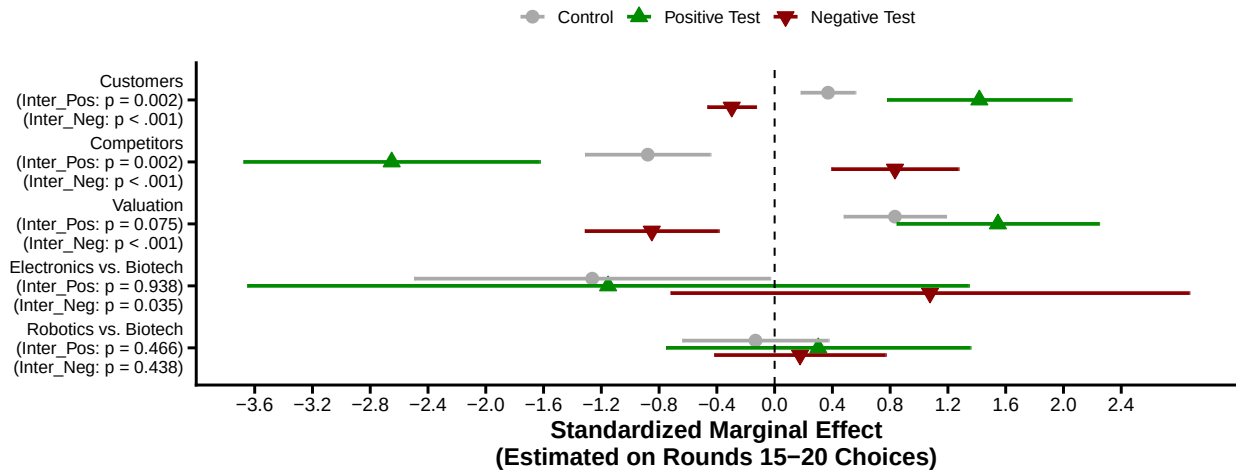
ggplot(
  df_contrasts,
  aes(
    x = estimate, y = term_names,
    color = message_type, shape = message_type, fill = message_type
  )
) +
  geom_point(size = 3, position = position_dodge(width = -.35)) +
  geom_errorbarh(
    aes(
      xmin = estimate - 1.96 * std.error,
      xmax = estimate + 1.96 * std.error
    ),
    height = 0, linewidth = 1,
    position = position_dodge(width = -.35)
  )

```

```

) +
labs(
  title = element_blank(),
  x = "Standardized Marginal Effect\n(Estimated on Rounds 15-20 Choices)",
  y = element_blank(),
  color = element_blank(),
  shape = element_blank(),
  fill = element_blank()
) +
theme_matplotlib() +
scale_x_continuous(breaks = seq(-36, 26, 4) / 10) +
geom_vline(xintercept = 0, linetype = "dashed") +
theme(
  axis.text.y = element_text(size = 10),
  axis.text.x = element_text(size = 11),
  legend.position = "top",
  panel.grid.major.y = element_blank(),
  panel.grid.minor.y = element_blank(),
  legend.background = element_rect(
    fill = "white",
    color = "white",
    linewidth = 0
  )
) +
scale_y_discrete(limits = rev) +
scale_color_manual(values = c("darkgrey", "green4", "red4")) +
scale_fill_manual(values = c("darkgrey", "green4", "red4")) +
scale_shape_manual(values = c(21, 24, 25))

```



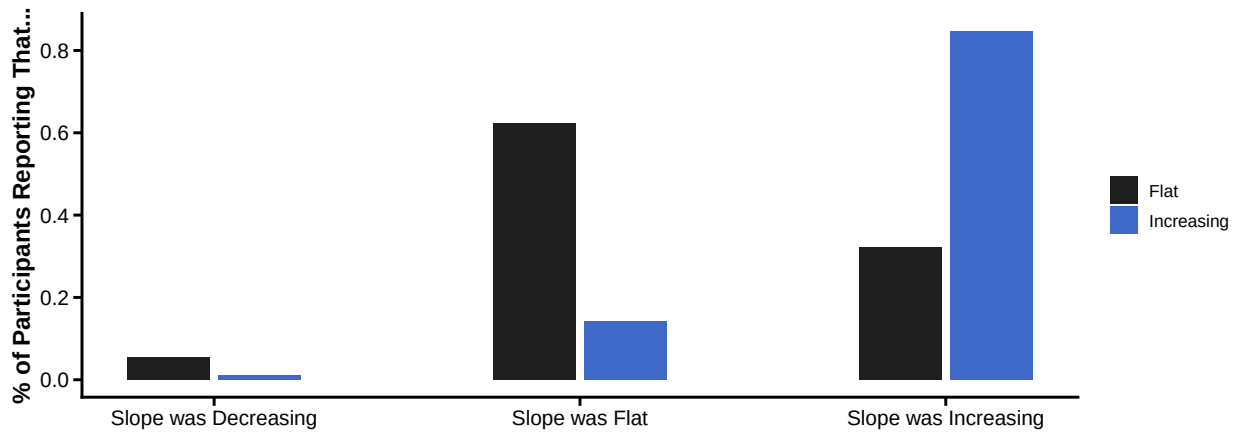
For participants in the “Control” and “Positive Test” conditions, the Number of Customers and Market Valuation are significant predictors of a project being chosen, and the Number of Competitors is a negative predictor. For participants in the “Negative Test” condition, the pattern reverses: these participants shift their selections toward attributes they had previously ignored, consistent with the adoption of a negative testing strategy.

4 Belief Regarding the Slope

Finally, participants were asked to identify the slope condition they were assigned to.

```
survey_data |>
  group_by(condition) %>%
  count(Slope) %>%
  mutate(pct = n / sum(n)) %>%
  ggplot(aes(x = Slope, y = pct, fill = condition)) +
  geom_col(position = position_dodge2(), width = .5) +
  scale_fill_manual(values = c("#1f1e1e", "#3f69c9")) +
  scale_x_continuous(
    breaks = c(-1, 0, 1),
    labels = c("Slope was Decreasing", "Slope was Flat", "Slope was Increasing")
  ) +
  theme_matplotlib() +
  labs(
    x = element_blank(),
    y = "% of Participants Reporting That...",
  )
```

```
fill = element_blank()
)
```



Most participants correctly identified their assigned condition: They realized that the “Increasing” condition had an increasing exogenous slope, and that the “Flat” condition had no slope.

5 Slope Beliefs by Prompt Type (Increasing Condition)

Among participants in the “Increasing” condition, we further examine whether slope beliefs differed by prompt type.

```
library(Hmisc)
survey_data %>%
  dplyr::filter(condition == "Increasing") %>%
  mutate(said_increasing = as.numeric(Slope == 1)) %>%
  ggplot(aes(x = message_type, y = said_increasing, color = message_type)) +
  stat_summary(fun.data = mean_cl_boot, geom = "pointrange", size = .8) +
  scale_color_manual(values = c("darkgrey", "green4", "red4")) +
  scale_y_continuous(
    labels = scales::label_percent(),
    limits = c(0, 1)
  ) +
  theme_matplotlib() +
  labs(
    x = element_blank(),
    y = "% Reporting the Slope Was Increasing",
```

```
color = element_blank()  
) +  
theme(legend.position = "none")
```

